Unit No. 1

Fundamental of Client-Side Scripting

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Course Outcome

- CO1 - Create interactive web pages using program flow control structure.

Content:

1.1 Introduction to the Scripting: Basic web architecture, Role of the client and server, Static vs. dynamic web pages

1.2 History of Scripting Technologies: HTML as a foundation, Early use of inline scripting, Limitations of static HTML, JavaScript

1.3 Features of JavaScript

1.4 Object Name, Property, method, Dot syntax, main event.

1.5 Values and Variables

1.6 Operators and Expressions – Primary Expressions, Object and Array initializers, function definition expression, property access expressions, invocation expressions, If Statement, if…else, if.. elseif, nested if statement. Switch…case statement, Loop statement – for loop, for…in loop, while loop, do…while loop, continue statement.

1.7 Querying and setting properties and deleting properties, property getters and setters.

Fundamental of Client-Side Scripting

After completing this unit students will be able to understand-

- ✓ Understand client-side scripting
- ✓ Differentiate Static and Dynamic web pages
- ✓ Understand JavaScript basics

- ✓ Write JavaScript programs using
  - Variables
  - Operators
  - Conditions
  - Loops
  - Objects
- ✓ Create interactive webpages

1.1 Introduction to the Scripting

What is Scripting?

Scripting is the process of writing small programs (called scripts) that make a web page interactive and dynamic.

In Simple Words

A script is a set of instructions that tells the browser what to do when a user interacts with a webpage.

Examples

- Display a welcome message
- Validate login form
- Calculate marks
- Show current date and time
- Open a popup window

Key Point

JavaScript is the most popular client-side scripting language used to create interactive webpages.

Why Do We Need Scripting?

Without scripting, webpages can only display information.

With scripting, webpages can:

✓ Respond to user actions

✓ Perform calculations

✓ Validate user input

✓ Display messages

✓ Update content without reloading the page

Real-Life Example

Without JavaScript

Clicking a button does nothing.

With JavaScript

Clicking the button displays:

"Welcome to MIT-WPU!"

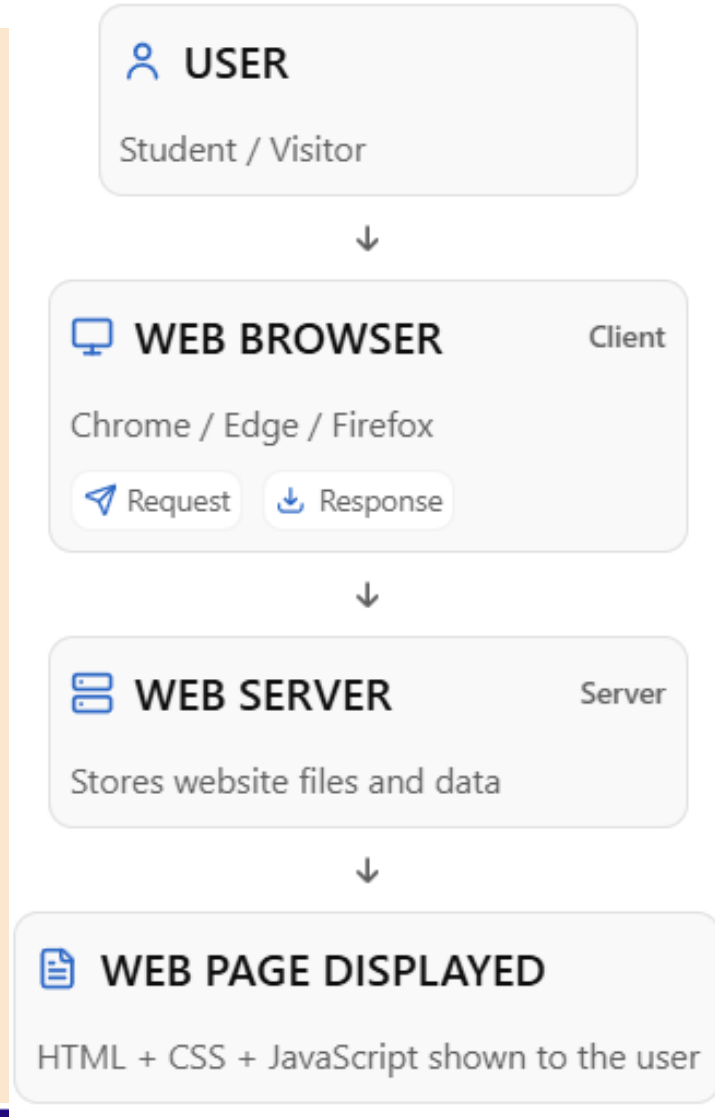
Basic Web Architecture

What is Web Architecture?

Web Architecture is the structure that shows how a user communicates with a website through a web browser and a web server.

Working Process

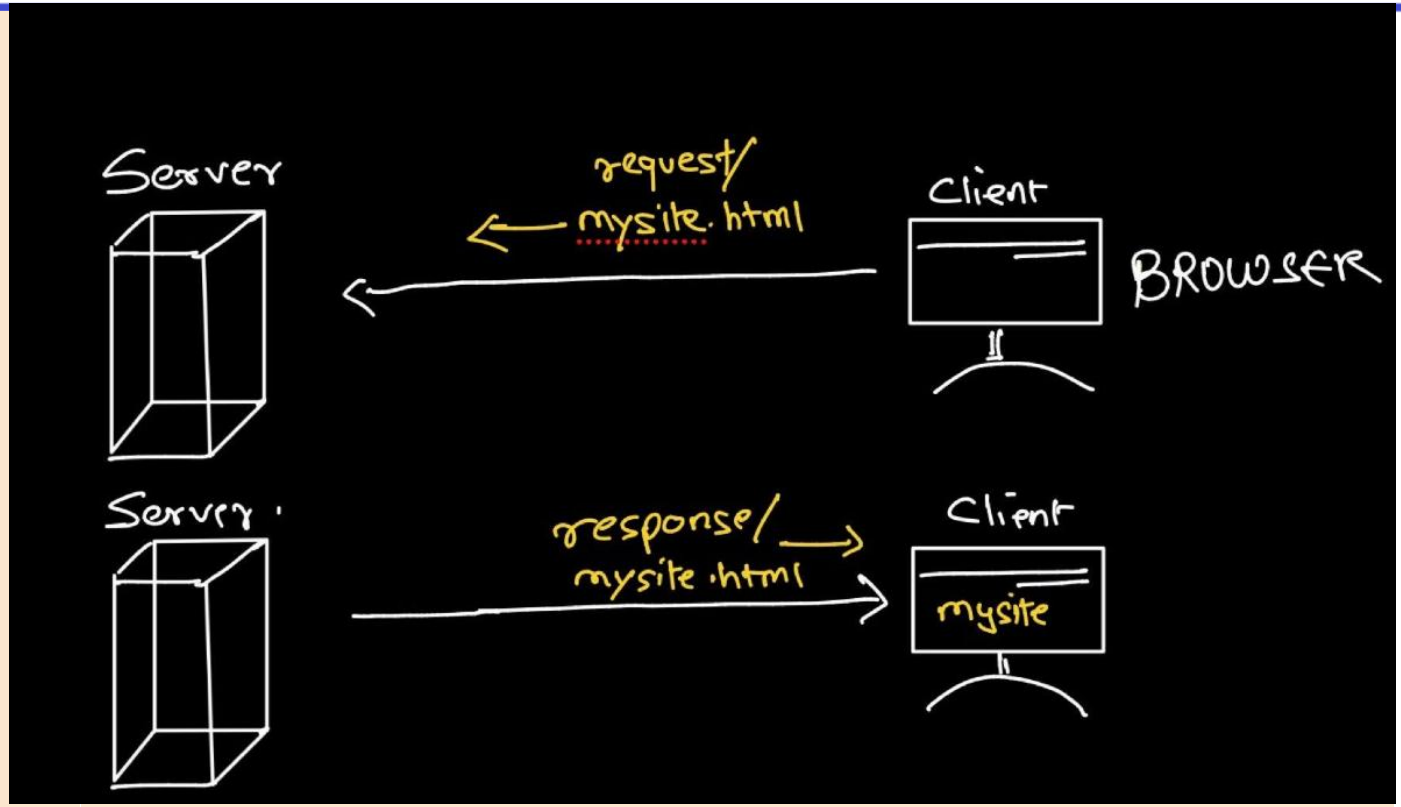
1. User enters a website address (URL) in the browser.
2. The Web Browser (Client) sends a request to the Web Server.
3. The Web Server processes the request and retrieves the required data.
4. The Server sends the response (HTML, CSS, JavaScript, images, etc.) back to the browser.
5. The Browser displays the webpage to the user.
6. JavaScript runs in the browser to make the webpage interactive.



Basic Web Architecture

Example: Opening Google

1. You type www.google.com in Chrome.
2. Browser sends a request to Google's server.
3. The server finds the Google homepage.
4. The server sends a response back.
5. The browser displays the page.
6. JavaScript then makes the page interactive (search suggestions, buttons, animations, etc.).



Role of the Client and Server

Role of the Client

Definition:

A Client is the user's device (computer, laptop, or mobile) that requests information from a web server through a web browser.

Responsibilities of the Client:

- Sends a request to the server.
- Displays the web page to the user.
- Accepts user input (keyboard, mouse, touch).
- Executes JavaScript in the browser.
- Displays images, videos, and animations.

Role of the Client

Definition:

A Client is the user's device (computer, laptop, or mobile) that requests information from a web server through a web browser.

Responsibilities of the Client:

- Sends a request to the server.
- Displays the web page to the user.
- Accepts user input (keyboard, mouse, touch).
- Executes JavaScript in the browser.
- Displays images, videos, and animations.

Examples:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari

Role of the Server

Definition:

A Server is a powerful computer that stores websites and provides web pages or data when requested by the client.

Responsibilities of the Server:

- Stores website files and databases.
- Receives requests from clients.
- Processes the requests.
- Sends the required web page or data back to the client.
- Handles multiple users at the same time.

Client vs Server

Client	Server
Requests information	Provides information
Runs in the user's browser	Runs on a remote computer
Displays web pages	Stores web pages and data
Executes JavaScript	Processes server-side programs

Client vs Server Real-Life Example

Student → Librarian

Student (Client): Requests a book.

Librarian (Server): Finds the book and gives it to the student.

Similarly:

Browser (Client): Requests a webpage.

Web Server: Sends the requested webpage back to the browser.

👉 Client = Requests Data
👉 Server = Provides Data

Static Web Pages

What is a Static Web Page?

A Static Web Page always displays the same content to every user.

Characteristics

1. Fixed content
2. Simple HTML and CSS
3. No database
4. No user interaction
5. Fast loading

Examples

- College notice page
- Company information page
- About Us page

Dynamic Web Pages

What is a Dynamic Web Page?

A Dynamic Web Page changes its content based on the user or their actions.

Characteristics

1. Interactive
2. User-specific content
3. Uses JavaScript
4. Can connect to a database
5. Content changes automatically

Examples

- Gmail
- Facebook
- YouTube
- Amazon
- Online Banking

Static vs Dynamic Web Pages

Static Web Page	Dynamic Web Page
Fixed content	Changing content
Same for every user	Different for different users
Uses HTML & CSS	Uses HTML, CSS & JavaScript
No database	Can use a database
Less interactive	Highly interactive

Dynamic Web Pages

What is a Dynamic Web Page?

A Dynamic Web Page changes its content based on the user or their actions.

Characteristics

1. Interactive
2. User-specific content
3. Uses JavaScript
4. Can connect to a database
5. Content changes automatically

Examples

- Gmail
- Facebook
- YouTube
- Amazon
- Online Banking

HISTORY OF SCRIPTING TECHNOLOGIES

How Web Technologies Evolved

Initially, websites were created using **HTML** only. As users wanted interactive and dynamic websites, new scripting technologies were introduced.

1991 — HTML

Static Web Pages

Web pages can only display information.

1995 — JavaScript

Interactive Web Pages

Users can interact with web pages.

Today — Modern Web Applications

Modern Web Applications

Web apps are dynamic, real-time and user-friendly.

KEY POINT: Web technology evolved from simple static pages to interactive and dynamic web applications.

HTML AS A FOUNDATION

What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to create the structure of a web page.

HTML is used to create:

- Headings
- Paragraphs
- Images
- Links
- Tables
- Forms

Example

```
<h1>Welcome to MIT-WPU</h1>
<p>This is my first
 web page.</p>
```

Welcome to MIT-WPU

This is my first web page.

Why is HTML called the Foundation?

- It provides the basic structure of every web page.
- Other technologies like CSS and JavaScript work together with HTML.

REMEMBER:

HTML (Builds Structure) + CSS (Adds Style) + JS (Adds Interactivity)

HTML Alone is Not Enough

Why not?

What HTML Can Do

- Display text
- Display images
- Create tables
- Create hyperlinks

What HTML Cannot Do

- Perform calculations
- Validate user input
- Respond to button clicks
- Update content automatically
- Display interactive messages

Example

A login page made only with HTML cannot check whether the password is correct.

Login (HTML only)

Username: student

Password: •••••

Login

No validation!

Early Use of Inline Scripting

<script>

What is Inline Scripting?

Inline scripting means writing JavaScript directly inside an HTML page using the <script> tag.

Example

```
<button onclick="alert('Welcome!')">
    Click Me
</button>
```

Click Me

Welcome!

Features

- JavaScript code is written inside the HTML file.
- Easy for small programs.
- Suitable for beginners.
- Used to make web pages interactive.

Limitation

For large projects, mixing HTML and JavaScript makes the code difficult to read and maintain.

Limitations of Static HTML

Problems with Static HTML Pages4.

1. Same content is shown to every user.
2. No interaction with users.
3. Cannot perform calculations.
4. Cannot validate forms.
5. Requires manual updates to change content.
6. Cannot respond to user actions like button clicks.
7. Real-Life Example

A college notice page is static.

Every student sees the same notice, and it does not change based on who is viewing it.

NEED FOR JAVASCRIPT

Why Was JavaScript Introduced?

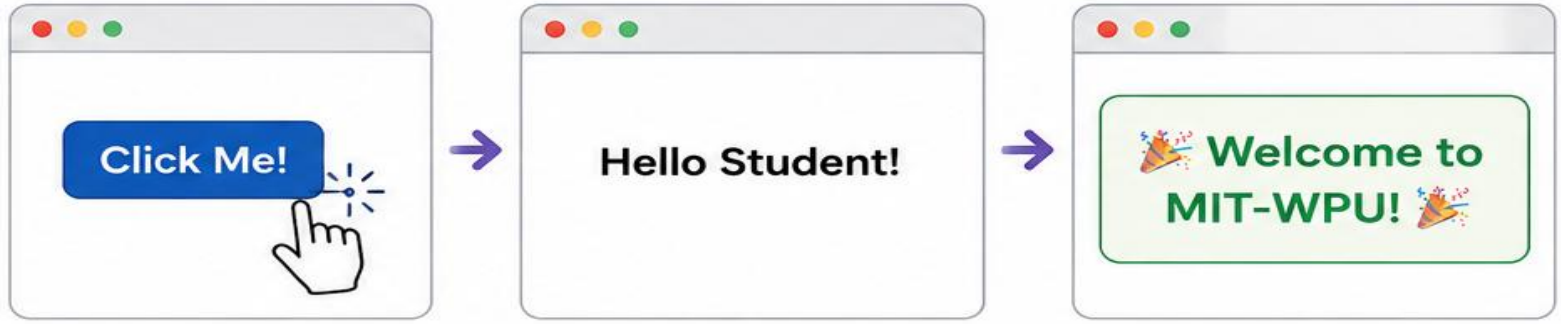
JavaScript was introduced to make web pages **interactive** and **dynamic**.

JAVASCRIPT CAN:

1. **Validate forms**
   Check user input before submitting.
2. **Perform calculations**
   Do math operations instantly.
3. **Respond to user actions**
   React to clicks, key presses, mouse movements, etc.
4. **Display pop-up messages**
   Show alerts, confirmations and messages.
5. **Change webpage content without reloading**
   Update parts of a page dynamically.
6. **Create animations and effects**
   Add animations, sliders, fading effects and more.

EXAMPLE

1. When you click a button,
2. Instead of simply displaying text,
3. JavaScript can display:



JS JavaScript makes the web pages **SMART, INTERACTIVE** and **USER FRIENDLY.**

REAL LIFE EXAMPLES

Login form validation | Calculators | Add to cart on click | Pop-up notifications | Live content updates | Animations and effects

KEY POINT: JavaScript turns a static web page into an **interactive** and **dynamic** experience for users.

1 NEED FOR JAVASCRIPT

Why Was JavaScript Introduced?

JavaScript was introduced to make web pages **interactive** and **dynamic**.

JAVASCRIPT CAN:

Validate forms
Check user input before submitting.

Perform calculations
Do math operations instantly.

Respond to user actions
React to clicks, key presses, mouse movements, etc.

JavaScript makes web pages **SMART, INTERACTIVE** and **USER FRIENDLY.**

2 INTRODUCTION TO JAVASCRIPT

What is JavaScript?

JavaScript (JS) is a client-side scripting language used to make web pages interactive, dynamic, and user-friendly.

In Simple Words

JavaScript adds intelligence and interactivity to a web page.

Without JavaScript:

Web pages only display information.

With JavaScript:

Web pages can respond to user actions.

EXAMPLES

Login validation

Calculator

Image slider

Online quiz

Shopping cart

Pop-up messages

3 WHAT IS A CLIENT-SIDE SCRIPTING LANGUAGE?

Client-Side Scripting

A client-side scripting language runs inside the user's web browser instead of on the web server.

User Clicks Button

Web Browser
(Chrome, Edge, Firefox)

Runs JavaScript

Shows Result Immediately

ADVANTAGES

- Faster execution
- No need to contact the server for every action
- Better user experience
- Reduces server load

4 HISTORY OF JAVASCRIPT

Development of JavaScript

Year	1995 JavaScript was created
Developer	Brendan Eich
Company	Netscape Communications
Development Time	About 10 days
Purpose	To make web pages interactive

Brendan Eich

Interesting Fact

Initially, JavaScript was named Mocha, then LiveScript, and finally renamed JavaScript before its public release.

Mocha → LiveScript → JavaScript

1 WHERE DOES JAVASCRIPT RUN?

User

Web Browser
(Chrome, Edge, Firefox)

JS Runs JavaScript

Interactive Web Page

Browsers That Support JavaScript

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari
- Opera

Almost every modern web browser supports JavaScript by default.

2 FEATURES OF JAVASCRIPT

1 Lightweight

Small and efficient language.
Executes quickly.

Example: Opening a calculator on a webpage.

2 Easy to Learn

Simple syntax.
Similar to English.

Example:

```
alert("Welcome");
```

3 Fast Execution

Runs directly in the browser.
No compilation required.

Example: Button click shows the result instantly.

4 Platform Independent

Works on Windows, Linux, macOS, Android, and iOS.

Any browser with JS support can run the same code.

5 Interpreted Language

Executes line by line.
No separate compilation step is needed.

6 Object-Based

Uses objects such as Date, Math, and Array.
Helps organize and reuse code.

7 Event-Driven

Responds to user actions like:

Mouse click

Keyboard input

Form submission

Page loading

3 REAL-LIFE APPLICATIONS OF JAVASCRIPT

Application	How JavaScript is Used
Gmail	Form validation and dynamic updates
YouTube	Video controls and recommendations
Amazon	Add to Cart and product search
Google Maps	Zoom, drag, and interactive maps
Online Banking	Input validation and notifications
College Website	Student login and result display

4 ADVANTAGES OF JAVASCRIPT

Advantages

- Makes websites interactive.
- Executes quickly in the browser.
- Easy to learn and use.
- Reduces server requests.
- Supported by all major browsers.
- Can create games, animations, and web applications.

5 SUMMARY

JavaScript runs inside the browser.

It makes web pages interactive and dynamic.

It is fast, easy to learn, platform independent, and widely used.

Remember:
HTML builds structure, CSS adds style, and JavaScript adds interactivity.

Features of JavaScript

Feature	Meaning	Example
🪶 **Lightweight**	JavaScript is a small and efficient language that executes quickly in the browser.	A calculator on a webpage gives instant results.
📖 **Easy to Learn**	It has simple syntax and is similar to English, making it beginner-friendly.	alert("Welcome");
⚡ **Fast Execution**	JavaScript runs directly inside the web browser without compilation.	A button click immediately shows a message.
🌍 **Platform Independent**	The same JavaScript code works on Windows, Linux, macOS, Android, and iOS.	A website behaves the same in Chrome on Windows and Android.

Features of JavaScript

Feature	Meaning	Example
🔍 **Interpreted Language**	JavaScript executes code line by line without a separate compilation step.	Errors are shown while the code is running.
🧩 **Object-Based**	JavaScript provides built-in objects like Date, Math, and Array to simplify programming.	Math.max(10,20) returns the larger value.
🖱️ **Event-Driven**	JavaScript responds to user actions such as clicks, typing, form submission, and page loading.	Clicking a **Submit** button validates a form.

1. INTRODUCTION TO JAVASCRIPT OBJECTS

What is an Object?

A JavaScript Object is a real-world entity that has:

- Properties → Characteristics or information about the object.
- Methods → Actions that the object can perform.

In simple words:
Object = Properties + Methods

Real-Life Examples

Object	Properties (What it has)	Methods (What it does)
Car	Color, Brand, Speed	Start(), Stop(), Brake()
Mobile Phone	Brand, Color, Price	Call(), Camera(), Lock()
Student	Name, Roll No, Branch	Study(), AttendClass(), WriteExam()

Key Point

- Every object has information (properties) and actions (methods).
- JavaScript uses objects to represent real-world things.

Visual Representation

Mobile Phone

Brand : Samsung
Color : Black
Price : ₹20,000

- Call()
- Camera()
- Lock()
- Play Music()

2. UNDERSTANDING OBJECTS IN JAVASCRIPT

How is an Object Created in JavaScript?

```
let mobile = {
    brand: "Samsung",
    color: "Black",
    price: 20000,

    call: function() {
        alert("Calling...");
    }
};
```

Remember

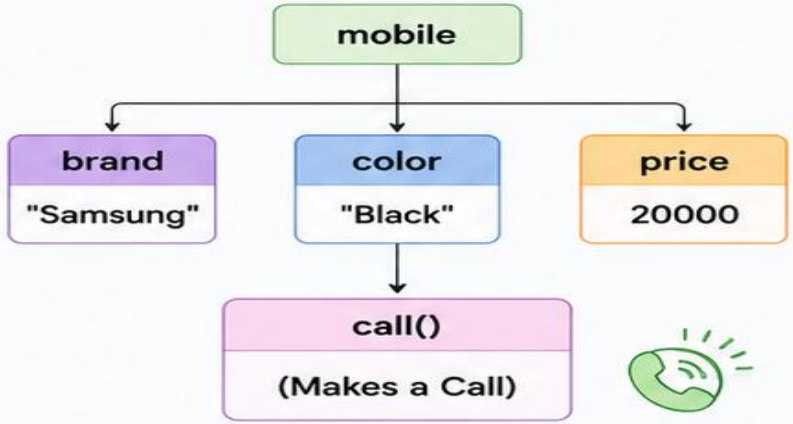
- Object → Represents a real-world thing.
- Properties → Describe the object.
- Methods → Perform actions.

Explain the Code

Code	Meaning
mobile	Object Name
brand	Property
color	Property
price	Property
call()	Method

An object is like a box that contains data (properties) and functions (methods)!

Visual Connection



Objects make our code organized, reusable and closer to real-world objects.

OBJECT, PROPERTY AND METHOD

EXAMPLE: STUDENT OBJECT 🎓

Student

Name	:	Rahul
Roll No	:	101
Branch	:	CSE

METHODS

- Study()
- WriteExam()
- AttendClass()

EXPLANATION

Term	Meaning	Example
Object	Real-world entity	Student
Property	Information about the object	Name, Roll No
Method	Action performed by the object	Study(), WriteExam()

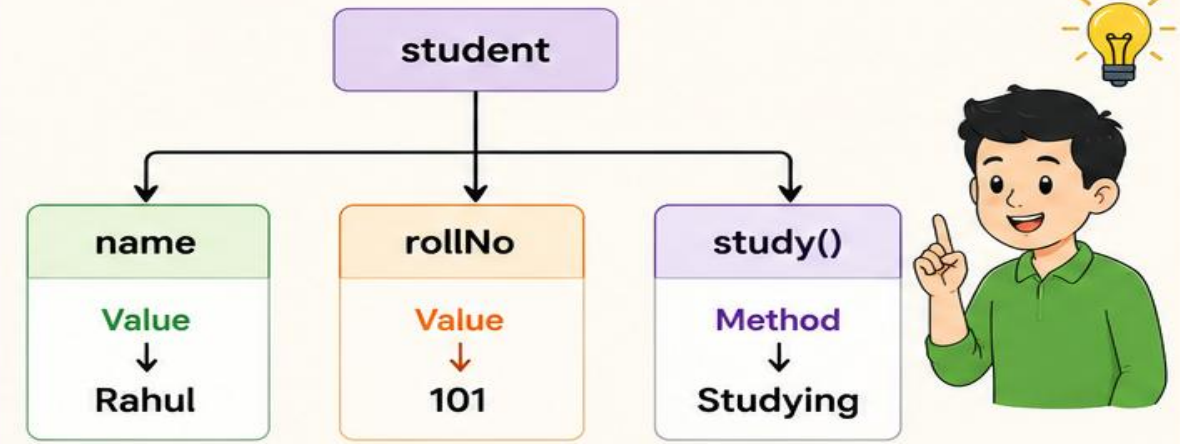
JAVASCRIPT OBJECT EXAMPLE

```
let student = {
    name : "Rahul",
    rollNo : 101,
    study : function(){
        alert("Studying");
    }
};
```

- student → Object Name
- name, rollNo → Properties
- study() → Method

Here, student is an object which has properties (name, rollNo) and a method (study()).

VISUAL REPRESENTATION



Properties store information about the object.
Methods define the actions or behavior of the object.

⭐ DOT SYNTAX ⭐

What is Dot Syntax?

Dot syntax is used to access the properties and methods of an object.

SYNTAX

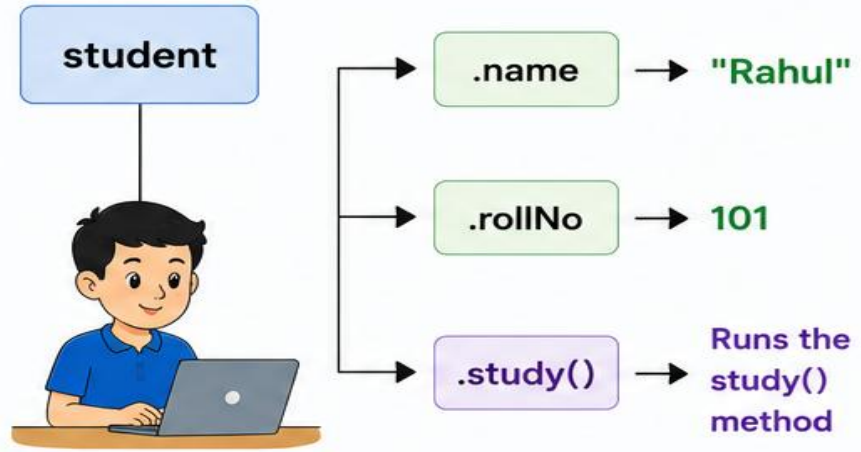
objectName.property	→	Access Property
objectName.method()	→	Call Method

EXAMPLE

```
student.name
student.rollNo
student.study()
```

Remember:
Use dot (.) to connect the object with its property or method.

VISUAL REPRESENTATION



WHY DOT SYNTAX?

WITHOUT DOT SYNTAX

JavaScript cannot identify which object's property you want.

WITH DOT SYNTAX

- Accesses object data easily.
- Calls methods easily.

MORE EXAMPLES

car.color	→	Accesses color property of car
car.start()	→	Calls start() method of car
mobile.camera()	→	Calls camera() method of mobile
student.name	→	Accesses name property of student

Key Point

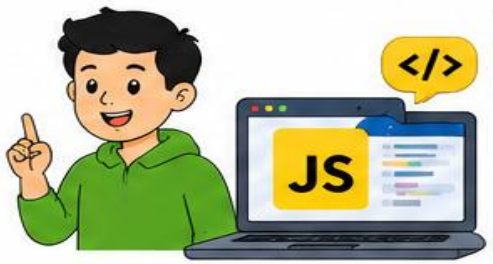
Dot syntax makes our code short, clean and easy to read.

SUMMARY Dot (.) links the object with its properties and methods. → Makes code simple and powerful!

JAVASCRIPT EVENTS

1. What is an Event?

An Event is an action or occurrence that happens on a web page. It can be performed by the user or the browser, and JavaScript responds to that action.



In Simple Words

Event = Something Happens → JavaScript Responds

Real-Life Examples

Real Life	JavaScript Event
Doorbell rings	Someone opens the door
Phone rings	You answer the call
User clicks a button	JavaScript performs an action

Visual Representation

EVENT
(User Action / Browser Action)
↓
JavaScript Detects Event
↓
Performs Required Action

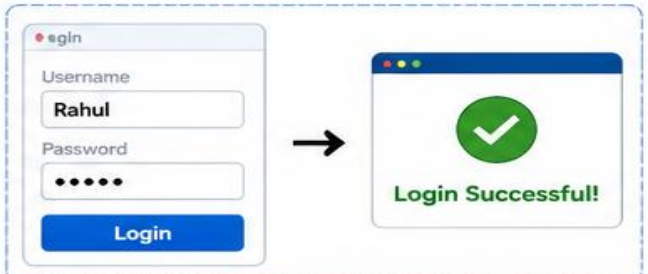
Key Point

JavaScript makes a webpage interactive by responding to events.

2. How JavaScript Events Work?

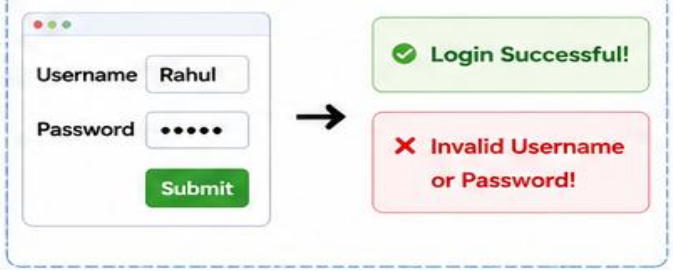
Event Flow (Login Button Example)

1. User
2. Clicks Login Button
3. JavaScript Event (onclick)
4. Checks Username & Password
5. Displays "Login Successful"



Another Example (Login Form)

1. Student fills Login Form
2. Clicks Submit Button
3. JavaScript Checks
   - Username
   - Password
4. Valid or Not?
5. Displays Result



Explanation

- JavaScript waits for an event.
- When the event occurs, JavaScript immediately performs the required task.

⭐ COMMON JAVASCRIPT EVENTS ⭐

Event	When It Happens	Real-Life Example
onclick	User clicks an element	SUBMIT — Click Submit button
ondblclick	User double-clicks	Open an image
onmouseover	Mouse moves over an element	Hover Me — Button changes color
onmouseout	Mouse leaves an element	Hover Me — Color returns to normal
onchange	Value changes	Select Country — Select a country from a dropdown
onkeydown	User presses a key	•••••• — Password typing
onload	Page finishes loading	Welcome! — Welcome message appears
onsubmit	Form is submitted	Login — Login or Registration form

EASY MEMORY TRICK

MOUSE EVENTS
- onclick
- ondblclick
- onmouseover
- onmouseout

KEYBOARD EVENT
- onkeydown

PAGE EVENT
- onload

FORM EVENTS
- onchange
- onsubmit

Remember:
Events = User/Browser Action → JavaScript Responds

EVENT EXAMPLES

1 onclick Event

```
<button onclick="alert('Welcome!')">
    Click Me
</button>
```

Click Me

OUTPUT

User → Clicks Button → onclick Event → JavaScript Runs → "Welcome!"

2 onmouseover Event

```
<p onmouseover="this.style.color='red'">
    Move Mouse Here
</p>
```

Move Mouse Here ⇢ Move Mouse Here

OUTPUT

The text changes to red when the mouse pointer moves over it.

3 onchange Event

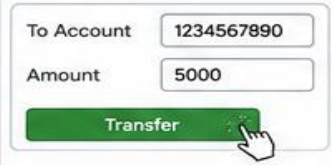
```
<select onchange="alert('Country Selected')">
  <option value="">Select Country</option>
  <option>India</option>
  <option>USA</option>
  <option>Japan</option>
</select>
```

Select Country → India

OUTPUT

An alert appears when a different country is selected.

REAL-LIFE APPLICATIONS OF EVENTS

Website	Event Used	How It Works (Example)
Gmail	onsubmit (Login Form)	Form submitted after entering email & password.
Amazon	onclick (Add to Cart)	Product added to the cart when button is clicked.
YouTube	onclick (Play/Pause Video)	Video plays or pauses when button is clicked.
Google Search	onkeydown (Typing Search)	Suggestions appear as keys are typed.
College Website	onchange (Select Department)	Department info changes when selection changes.
ATM Machine	onclick (Press Number buttons)	Numbers are entered when buttons are clicked.
Online Banking	onsubmit (Transfer Money)	 Money transfer done after form submission.

REMEMBER: Events make websites interactive! → **User Action** (Event Occurs) → **JavaScript Responds** → **Webpage Reacts** (Action Performed)

SUMMARY

JavaScript Events at a Glance

User Action — User performs an action (click, type, move mouse, etc.)

↓

JavaScript Event — JavaScript detects the event

↓

JavaScript Code Executes — The associated JavaScript code runs

↓

Web Page Responds — Web page updates or responds to the action

REMEMBER

Event
→ Something happens.

JavaScript
→ Detects the event.

Action
→ JavaScript executes the code.

JavaScript Events make web pages **INTERACTIVE** and **USER-FRIENDLY**!

1.5 Values and Variables

🎯 Learning Outcome

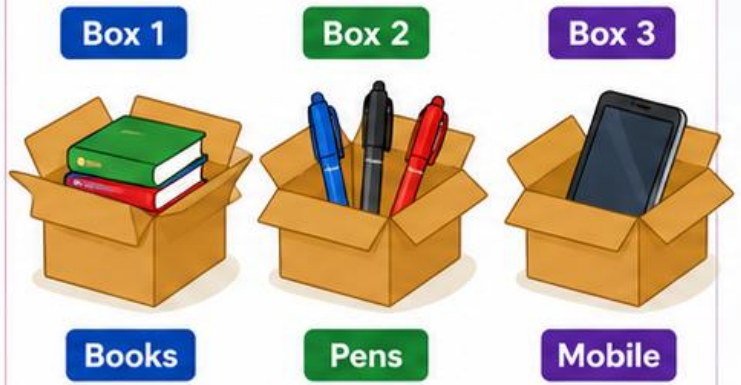
By the end of this topic, you will be able to:

- Understand Values and Variables
- Store data in variables
- Use of variables in JavaScript programs

VALUES AND VARIABLES

THINK ABOUT IT!

Imagine you have three boxes.



Each box stores a different item.

Similarly, a variable is like a box that stores **data (values).**

1 WHAT IS A VALUE?

A **Value** is the actual information or data stored in a program.

Examples of Values

25	100	"Rahul"
"MIT-WPU"	true	false

Real-Life Example

- Student Name = Rahul
- Age = 19
- Marks = 85
- Passed = true

Key Point
A Value is the data that we want to store and use in a program.

2 WHAT IS A VARIABLE?

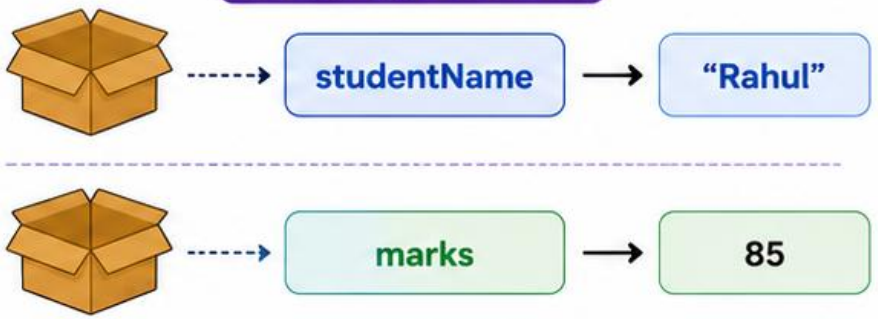
A **Variable** is a named memory location used to store a value.

Formula

Variable = Container (Box)

Value = Item stored inside the Box

Visual Representation



Key Point
Variables help us store, update, and reuse data.

Variables make our programs **organized**, **easy to read**, and **easy to maintain**.

Store once, use many times!

Understand today, code better tomorrow!

Image created by- *Prof. Priti Chavhan*

3 WHY DO WE NEED VARIABLES?

Without Variables ❌

Rahul

85

Pune

19

☹ Difficult to understand what each value means.

With Variables ✅

```
let studentName = "Rahul";
let marks = 85;
let city = "Pune";
let age = 19;
```

☺ Now the program is easy to read and understand.

4 DECLARING VARIABLES IN JAVASCRIPT

JavaScript provides three keywords to create variables.

Keyword	Meaning	Used Today?
let	Variable value can change	✅ Yes
const	Fixed value (cannot change)	✅ Yes
var	Old method	⚠ Avoid in new programs

Example

```
let name = "Rahul";
const college = "MIT-WPU";
let marks = 85;
```

5 NAMING RULES FOR VARIABLES

✅ Correct Variable Names	❌ Incorrect Variable Names
✓ studentName	✗ 1name
✓ marks	✗ student-name
✓ totalMarks	✗ class
✓ firstName	✗ my name
✓ age	✗ @marks

Rules

- Start with a letter, _, or $.
- Do not start with a number.
- Do not use spaces.
- Variable names are case-sensitive.

Example:

name

Name

NAME

These are three different variables.

6 REAL-LIFE EXAMPLE – STUDENT INFORMATION

```
let studentName = "Rahul";
let age = 19;
let branch = "Computer";
let marks = 90;
let passed = true;
```

- studentName → Student Name
- age → Student Age
- branch → Student Branch
- marks → Student Marks
- passed → Student Status

7 OUTPUT

```
Name   : Rahul
Age    : 19
Branch : Computer
Marks  : 90
Passed : true
```

Variables help us store, update and reuse data easily in our programs!

JS

Variables make code readable and meaningful.

Easy to update and use again.

Important building blocks of JavaScript programs!

Image created by- Prof. Priti Chavhan

1.2 Operators and Expressions

What is an Expression?

An expression is a combination of values, variables, and operators that produces a result.

General Form

```
variable = value operator value;
```

- **variable** — Name given to store result
- **value** — Data or value
- **operator** — Performs operation
- **value** — Another data or value

Examples

```
5 + 3        // 8      → 8
10 * 2       // 20     → 20
age > 18     // true   → true
```

Real-Life Example

Price = ₹100 − Discount = ₹20 = Final Price = ₹100 - ₹20 = ₹80

Remember: An expression always produces a value.

Expressions help us calculate, compare and make decisions in programs.

MIT-WPU POLYTECHNIC

1.2 Operators and Expressions

PRIMARY EXPRESSIONS

Primary expressions are the basic building blocks of JavaScript expressions.

They include:

1. Numbers
2. Strings
3. Boolean values
4. Variables
5. null
6. true / false

Examples

10
"MIT-WPU"
true
studentName
null

Practical Example

```
let marks = 85;
let college = "MIT-WPU";
let passed = true;
```

marks	85
college	"MIT-WPU"
passed	true

Values Stored in Variables

OBJECT INITIALIZER

An object initializer is used to create an object with properties and values.

Syntax

```
let objectName = {
  property : value
};
```

Example

```
let student = {
  name: "Rahul",
  age: 19,
  branch: "Computer"
};
```

Output

Student

Name	→	Rahul
Age	→	19
Branch	→	Computer

JavaScript Expressions start with Primary Expressions and help us build complex programs using Objects and more.

Image created by- *Prof. Priti Chavhan*

1.2 Operators and Expressions

1 What is an Array?

An Array is a special type of variable that stores **multiple values** under a single variable name.

2 Why Do We Need an Array?

Suppose you want to store marks of 5 subjects.

✕ Without an Array

```
let marks1 = 85;
let marks2 = 90;
let marks3 = 88;
let marks4 = 76;
let marks5 = 92;
```

Too many variables are needed, making the program difficult to manage.

✓ With an Array

```
let marks = [85, 90, 88, 76, 92];
```

All values are stored in one variable, making the program shorter and easier to understand.

3 Syntax

```
let arrayName = [value1, value2, value3];
```

General Form

```
Array Name = [Value1, Value2, Value3, ...]
```

4 Example

```
let colors = ["Red", "Blue", "Green"];
```

Visual Representation

colors

Red	Blue	Green
0	1	2

Note: Array elements are stored using index numbers, starting from 0.

5 Practical Example – Store Marks of Five Subjects

```
let marks = [85, 90, 88, 76, 92];
```

Visual Representation

marks

85	90	88	76	92
0	1	2	3	4

Accessing Values

```
marks[0]
// 85
```

```
marks[2]
// 88
```

```
marks[4]
// 92
```

6 Real-Life Example

Imagine a student attendance register.

Instead of creating five separate variables, JavaScript stores them in one array.

```
let students =
  ["Rahul", "Sneha",
  "Amit", "Priya",
  "Rohan"];
```

7 Advantages of Arrays

- Stores multiple values in a single variable.
- Reduces the number of variables.
- Makes programs shorter and easier to read.
- Data can be accessed using index numbers.
- Useful for storing marks, names, prices, products, and more.

8 Remember

Variable
Stores one value

```
marks = 85
```

VS

Array
Stores many values

```
marks = [85, 90,
88, 76, 92]
```

Arrays make your code organized, efficient and easy to manage!

Image created by- *Prof. Priti Chavhan*

1.6 Operators and Expressions

FUNCTION DEFINITION EXPRESSION

Function Definition

A function is a block of code that performs a specific task.

SYNTAX

```
let functionName = function(){
   code
};
```

`let` → declares the function variable

`function()` → creates a function

`code` → block of statements

EXAMPLE

```
let greet = function(){
   alert("Welcome");
};
```

Here,

- Function name: `greet`
- Task: Show "`Welcome`" message
- Function is stored in variable 'greet'

A function can be created using a **function expression** and stored in a variable.

CALLING FUNCTION

To execute (run) the function:

```
greet();
```

OUTPUT

The `alert()` method displays the message "**Welcome**".

1. **Define Function** (Write the code)
2. **Call Function** (Execute the code)
3. **Output** (Result shown)

1.6 Operators and Expressions

Property Access Expression

Use the **dot (.)** operator to access object properties.

Syntax

```
object.property
```

Example

```
student.name
student.age
```

```
let student = {
  name: "Rahul",
  age: 19,
  branch: "Computer"
};
```

Access Properties →

Output

```
student.name  →  Rahul
student.age   →  19
```

Think of it as opening a folder and reading one specific file.

Invocation Expression

Calling or executing a function is called **function invocation.**

Syntax

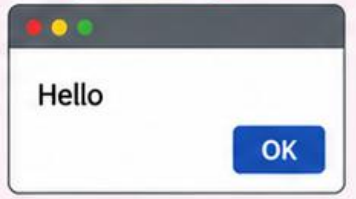
```
functionName();
```

Example

```
function greet(){
    alert("Hello");
}

greet();
```

Function Defined → `greet();` Function Invoked (Called) →

Output



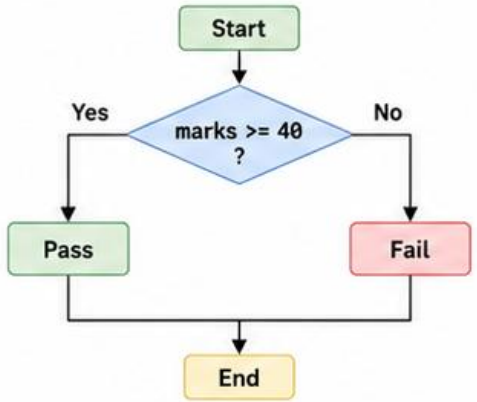
1.6 Operators and Expressions

1 if...else Statement

Chooses between two conditions.

Example

```
let marks = 45;
if(marks >= 40){
   document.write("Pass");
}
else{
   document.write("Fail");
}
```



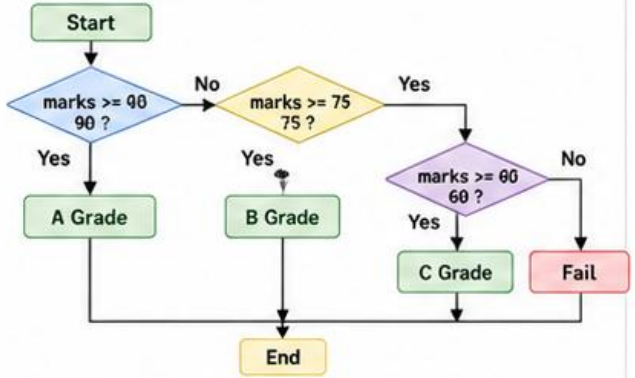
Output Pass

2 if...else if Ladder

Used when multiple conditions need to be checked.

Example

```
let marks = 82;
if(marks >= 90)
   document.write("A Grade");
else if(marks >= 75)
   document.write("B Grade");
else if(marks >= 60)
   document.write("C Grade");
else
   document.write("Fail");
```



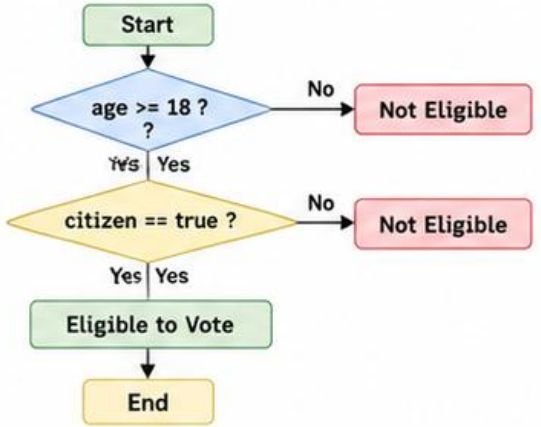
Output B Grade

3 Nested if Statement

A nested if means an if statement inside another if statement.

Example

```
let age = 20;
let citizen = true;
if(age >=18){
   if(citizen){
      document.write("Eligible to Vote");
   }
}
```



Output Eligible to Vote

4 switch...case Statement

Used when one variable has many possible values.

Syntax

```
switch(expression){
  case value:
     statements;
     break;
  default:
     statements;
}
```

Example

```
let day = 2;
switch(day){
  case 1:
     document.write("Monday");
     break;
  case 2:
     document.write("Tuesday");
     break;
  default:
     document.write("Invalid Day");
}
```

Output Tuesday

Master decision-making in JavaScript with if, else, and switch statements!

Image created by- *Prof. Priti Chavhan*

LOOP STATEMENTS

1. What is a Loop?

A loop is used to execute the same block of code multiple times until a condition becomes false.

2. Real-Life Example

Imagine a teacher asks you to write:

" Welcome to JavaScript "
10 times.

3. Without a Loop

```
Welcome to JavaScript
Welcome to JavaScript
Welcome to JavaScript
...
(repeat 10 times)
```

Writing the same line again and again is time-consuming and error-prone.

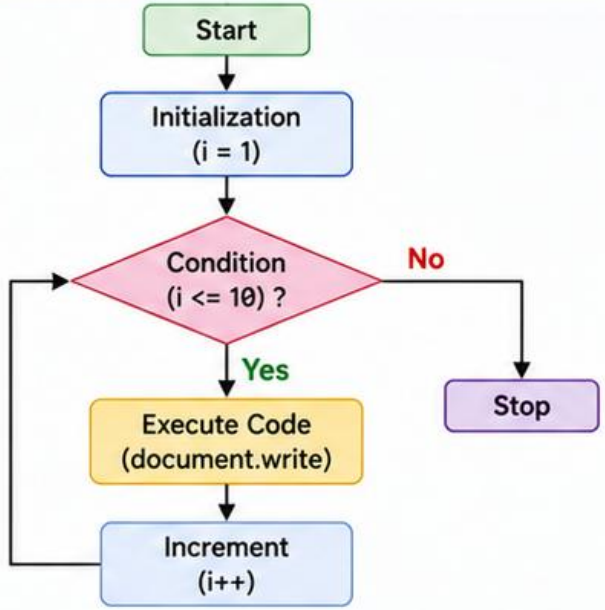
VS

4. With a Loop

```
for(let i = 1; i <= 10; i++){
    document.write("Welcome to JavaScript<br>");
}
```

The loop repeats the line automatically the required number of times.

5. How Loop Works (Flow Diagram)



6. Output

```
Welcome to JavaScript
Welcome to JavaScript
Welcome to JavaScript
...
(repeated 10 times)
```

10 Times

KEY POINT → **Loop** = **Repeat** the same task automatically.

JavaScript LOOPS

1 Types of Loops in JavaScript

JavaScript provides four commonly used loops:

Loop	Purpose
for	When the number of iterations is known
for...in	To access properties of an object
while	Repeats while the condition is true
do...while	Executes at least once before checking the condition

2 for Loop

What is a for Loop?

A for loop is used when we know **how many times** we want to repeat a task.

Syntax

```
for(initialization; condition; increment){
    statements;
}
```

Example

```
for(let i = 1; i <= 5; i++){
    document.write(i + "<br>");
}
```

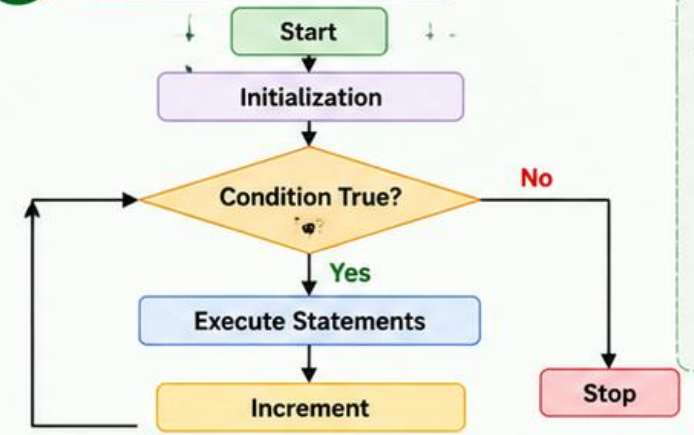
Output

1
2
3
4
5

Real-Life Example

Printing roll numbers from 1 to 5.

3 Flow of a for Loop



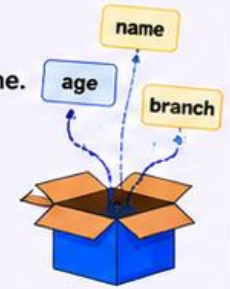
Remember:
A for loop first initializes, checks the condition, executes the code, increments the variable, and repeats.

Visual Analogy – For Loop

for...in Loop in JavaScript

1 What is a for...in Loop?

- A `for...in` loop is used to access or display all the properties (keys) of an object one by one.



✅ It automatically goes through each property of the object.

2 Why Do We Need a for...in Loop?

Without for....in Loop ❌

```
document.write(student.name);
document.write(student.age);
document.write(student.branch);
```

❗ If there are many properties, the code becomes lengthy.

With for....in Loop ✅

```
for(let key in student){
    document.write(key + " : " +
     student[key] + "<br>");
}
```

✅ JavaScript automatically displays all the properties.

3 Syntax

```
for(let key in object){
   statements;
}
```

Understanding the Syntax

Part	Meaning
`key`	Stores the property name (name, age, branch)
`in`	Goes through each property of the object
`object`	The object whose properties are accessed

4 Example

```
let student = {
    name: "Rahul",
    age: 19,
    branch: "Computer"
};
for(let key in student){
    document.write(key + " : " +
     student[key] + "<br>");
}
```

Output

```
name : Rahul
age : 19
branch : Computer
```

5 How Does It Work? (Step-by-Step Execution)

Iteration	key (property)	student[key] (value)	Output
1	name	Rahul	name : Rahul
2	age	19	age : 19
3	branch	Computer	branch : Computer

6 Visual Representation

student Object

```
name   → Rahul
age    → 19
branch → Computer
```

`for...in` visits each property one by one

```
name   → Rahul
age    → 19
branch → Computer
```

7 Practical Use

- ✅ Display student information
- ✅ Display employee details
- ✅ Show product information
- ✅ Display customer records

Student ID Card

```
Name   : Rahul
Age    : 19
Branch : Computer
```

8 Key Takeaway

for → Used for numbers and counting.

for....in → `for...in` → Used for objects and their properties.

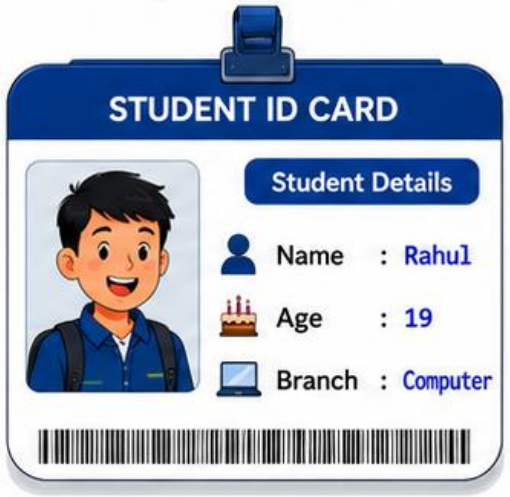
9 Quick Note

- `for...in` is used with objects.
- `key` stores the property name.
- `object[key]` gives the value of that property.

for...in Loop in JavaScript

Real-Life Example

Imagine a Student ID Card.



Instead of reading each detail manually,

👉 **for...in** reads every field one by one automatically.

Practical Applications

- Display student information
- Display employee details
- Show product information
- Read user profile data
- Display customer records

Remember

for Loop	for...in Loop
1 Used for numbers	1 Used for objects
2 Repeats a fixed number of times	2 Visits each property of an object
3 Example: `for(let i=1; i<=5; i++)`	3 Example: `for(let key in student)`

Quick Tip for Students

Remember this simple rule:

for → Used for numbers and counting. 1 2 3 4 5 6 ...

for...in → Used for objects and their properties.

```
let student = {
  name: "Rahul",
  age: 19,
  branch: "Computer"
};
for(let key in student){
  document.write(key + " : " + student[key] + "<br>");
}
```

Output

```
name   : Rahul
age    : 19
branch : Computer
```

continue Statement

What is a continue Statement?

The continue statement is used inside a loop to skip the current iteration and move directly to the next iteration.

👉 It does not stop the loop.

👉 It only skips a particular iteration.

```
for(initialization; condition; increment){

    if(condition)
        continue;

    statements;
}
```

1 Querying and Setting Properties

What are Object Properties?

Properties are the **characteristics** or **information** of an **object**.

Example

```
let student = {
    name : "Rahul",
    age  : 19,
    branch : "Computer"
};
```

Properties of Student Object

Property	Value
name	Rahul
age	19
branch	Computer

Remember:
Properties store information about an object.

2 Querying Properties

JS

What is Querying a Property?

Querying means **reading** or **accessing** the **value** of a property.

Syntax

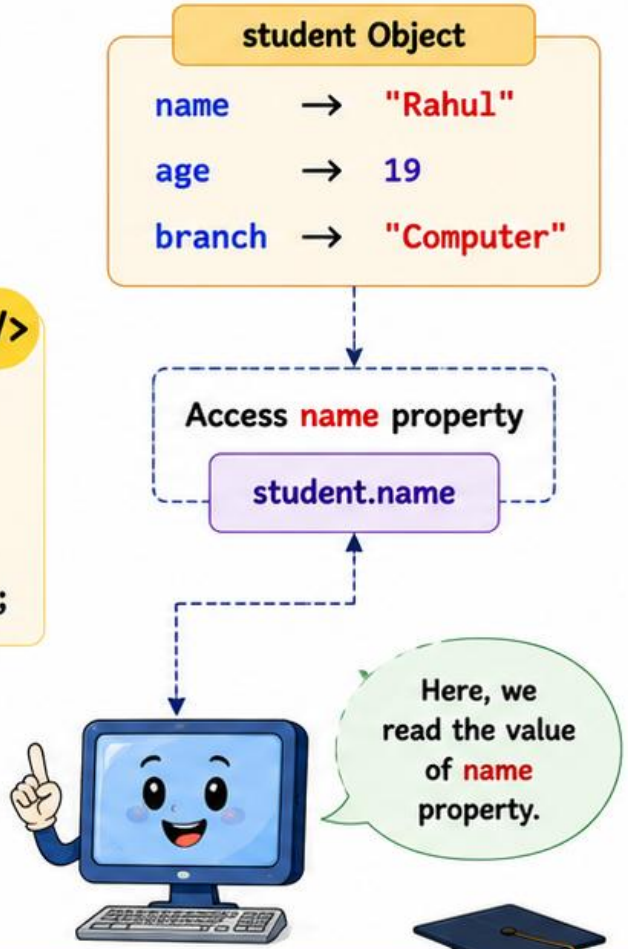
```
objectName.property
```

Example

```
let student = {
    name : "Rahul",
    age  : 19
};
document.write(student.name);
```

Output

Rahul



3 Setting Properties

What is Setting a Property?

Setting means **assigning** or **changing** the **value** of a property.

Syntax

```
objectName.property = value;
```

Example

```
let student = {
    name : "Rahul",
    age  : 19
};
student.age = 20;
```

Before	After
Student Object	Student Object
age → 19	age → 20

Visual Representation

student.age = 19	→	student.age = 20	→	student.age = 20
(Initial Value)		(Setting New Value)		(Updated Value)

Remember

Existing property values can be modified anytime.

Real-Life Example

Just like updating the **age** on a student ID card.

4 Adding New Properties

What is Adding New Properties?

JavaScript allows adding properties **dynamically**.

Example

```
let student = {
    name : "Rahul"
};
student.city = "Pune";
```

Output

```
name : Rahul
city : Pune
```

Visual Representation

Before Adding	After Adding
student Object	student Object
name → Rahul	name → Rahul
	city → Pune

Key Point

Objects are **flexible**. You can add new properties at any time without defining them earlier.

5. DELETING PROPERTIES

5.1 What is Deleting a Property?

Deleting means removing a property from an object permanently.

5.2 Syntax

```
delete objectName.property;
```

5.3 Example

```
let student = {
    name    : "Rahul",
    age     : 19,
    branch  : "Computer"
};

delete student.age;
```

5.4 Before and After Delete

Before Delete	
name	Rahul
age	19
branch	Computer

➡

After Delete	
name	Rahul
branch	Computer

5.5 Key Points

- Once a property is deleted, it no longer exists in the object.
- We use `delete` keyword to remove properties.
- Deleting a property does not give any error if the property does not exist.
- It reduces the size of the object.

Note: `delete` only removes the property from the object. It does not delete the value from memory immediately.

6. PROPERTY GETTERS

6.1 What is a Getter?

A getter is a special method used to read a property value.

6.2 Why Use Getters?

- Control how data is accessed.
- Improve readability of code.
- Provide security for data.
- Allow execution of code when property is read.

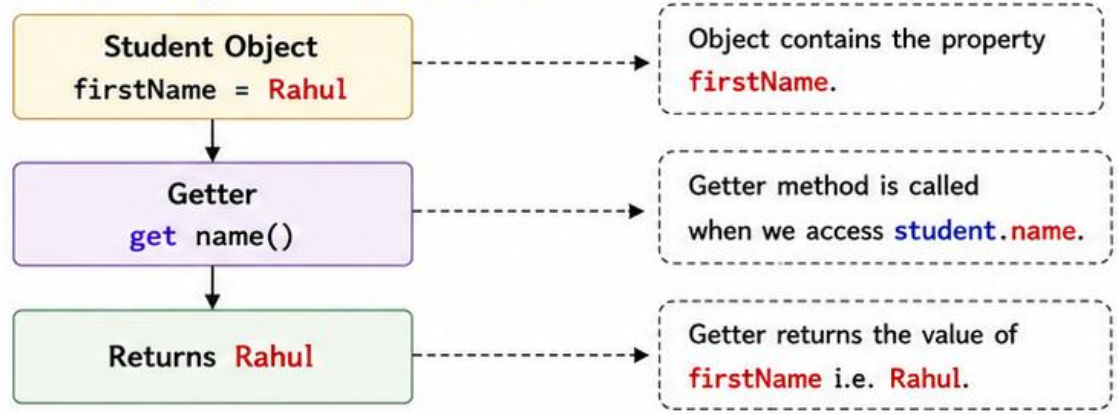
6.3 Example

```
let student = {
    firstName : "Rahul",
    get name(){
        return this.firstName;
    }
};
document.write(student.name);
```

6.4 Output

Rahul → (Value returned by getter)

6.5 Visual Representation of Getter



1. PROPERTY SETTERS

1.1 What is a Setter?

A setter is a special method used to change or update a property value.

1.2 Why Use Setters?

- Validate data before storing.
- Control changes to object properties.
- Improve data security.

1.3 Example

```
let student = {
     firstName : "",
     set name(value){
          this.firstName = value;
     }
};
```

```
student.name = "Rahul";
```

1.4 Output

```
firstName = Rahul
```

Note: The setter is called automatically when we assign a value to the property.

2. GETTER AND SETTER TOGETHER

2.1 Example

```
let student = {
     firstName : "",
     get name(){
          return this.firstName;
     },
     set name(value){
          this.firstName = value;
     }
};
```

```
student.name = "Rahul";     // calls setter
```

```
document.write(student.name);    // calls getter
```

2.2 Output

```
Rahul
```

2.3 How It Works?

Setter
`student.name = "Rahul"`

↓

Stores Value

↓

`firstName = Rahul`

↓

Getter
`student.name`

↓

Returns Rahul

Key Points

- Setter is used to set or update the value of a property.
- Getter is used to read or access the value of a property.
- Both together provide better control and security over data.
- They are widely used in modern JavaScript applications.

Thank You!

I hope this session has helped you to understand **basic concepts of Javascript** and its **basic usage.**

Keep Learning, Keep Coding!

Prepared by – *Prof. Priti Chavhan*

Unit No. 2

Array, Function and String

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Course Outcome

- Implement Arrays and functions using Java script.

Content:

2.1 Array – declaring an Array, Initializing an Array, defining an Array element. Looping an Array, Adding an Array element, sorting an Array element. Combining an Array element into a String, changing elements of an Array, Objects as associative Arrays.

2.2 Function – defining a function, writing a function, adding arguments, scope of variable and arguments.

2.3 Calling a function – calling a function with or without an argument, calling function from HTML, function calling another function. Returning of value from a function

2.4 String – manipulate a string, joining a string, retrieving a character from given position, retrieving a position of a character in a string, dividing text, copying a sub string, converting a string to a number and number to string , changing the case of string, finding a Unicode of a character-charCodeAt (), fromCharCode()

2.1 Arrays in JavaScript

Think About It!

Imagine a teacher wants to store the marks of 5 students.

Without an array:

```
let marks1 = 85;
let marks2 = 90;
let marks3 = 78;
let marks4 = 88;
let marks5 = 92;
```

Too many variables make the program difficult to manage.

With an array:

```
let marks = [85, 90, 78, 88, 92];
```

Only one variable stores all the values.

Definition

An **Array** is a special variable used to store multiple values under a single variable name.

Remember:

Variable → One Value

```
let age = 20;
```

Array → Multiple Values

```
let ages = [20, 21, 22];
```

01 Declaring an Array

What is Array Declaration?

Array declaration means creating an array variable.

Syntax

```
let arrayName;
```

Example

```
let students;
```

Visual

students

Empty

Key Point

Declaring an array creates the array variable, but it does not store any values.

02 Initializing an Array

What is Initialization?

Initialization means storing values in an array.

Syntax

```
let colors = ["Red", "Blue", "Green"];
```

Visual Representation

colors

Red	Blue	Green
0	1	2

Array indexing starts from 0.

2.1 Arrays in JavaScript

4 Defining (Accessing) Array Elements

What is an Array Element?

- Each value stored in an array is called an array element.
- Each element has an index number.

Example

```
let fruits = ["Apple", "Mango", "Orange"];
```

Access elements:

`fruits[0]` `fruits[1]` `fruits[2]`

Output

Apple

Mango

Orange

5 Looping Through an Array

Why Use a Loop?

A loop helps us display all array elements automatically.

Example

```
let marks = [85,90,78,88];
for(let i=0; i<marks.length; i++){
   document.write(marks[i] + "<br>");
}
```

Output

85
90
78
88

Real-Life Example

Displaying marks of all students.

6 Adding Array Elements

Adding New Elements

JavaScript provides the push() method.

Example

```
let colors = ["Red", "Blue"];
colors.push("Green");
```

Before

Red
Blue

After

Red
Blue
Green

push() adds an element at the end of the array.

7 Sorting Array Elements

Sorting an Array

Sorting arranges elements in ascending or alphabetical order.

Example (String)

```
let names = ["Rahul", "Amit", "Sneha"];
names.sort();
```

Output

Amit
Rahul
Sneha

Numeric Example

```
let marks = [90, 75, 85, 60];
marks.sort();
```


Numbers may require a compare function for proper numeric sorting.

```
marks.sort((a, b) => a - b);
```

Output: 60, 75, 85, 90

Prepared by – *Prof. Priti Chavhan*

2.1 Arrays in JavaScript

8 Combining Array Elements into a String

join() Method

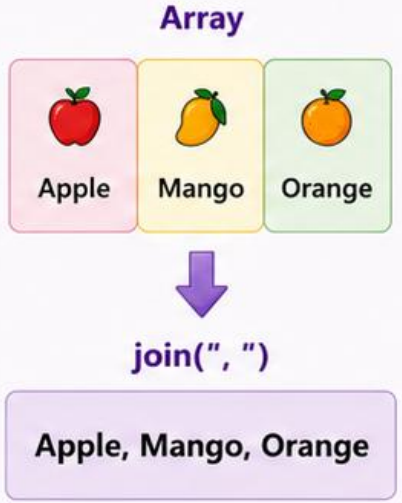
The join() method combines all array elements into a single string.

Example

```
let fruits = ["Apple", "Mango", "Orange"];
fruits.join(", ");
```

Output

Apple, Mango, Orange



Real-Life Use

Displaying a list of selected subjects.

Selected Subjects:

Maths, Physics, Chemistry

9 Changing Array Elements

Updating Elements

Array elements can be changed using their index.

Example

```
let colors = ["Red", "Blue", "Green"];
colors[1] = "Yellow";
```

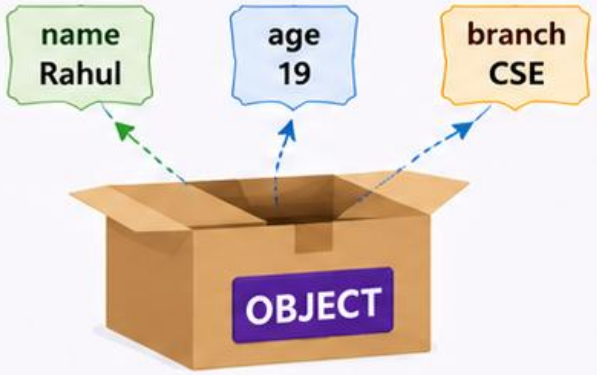
⭐ We changed the element at index 1 (Blue) to "Yellow".

2.1 Arrays in JavaScript

Objects as Associative Arrays

What is an Object?

An object stores information in the form of **key : value** pairs.



Unlike arrays, objects use **property names (keys)** instead of index numbers.

Example

```
let student = {
      name: "Rahul",
      age: 19,
      branch: "CSE"
};
```

Key (Property)		Value
name	→	Rahul
age	→	19
branch	→	CSE

Accessing Values

`student.name` --------→ Access the value of **name**

`student.age` --------→ Access the value of **age**

Output

`student.name` --------→ Rahul

`student.age` --------→ 19

Prepared by – *Prof. Priti Chavhan*

2.2 Functions in JavaScript

1 Think About It

Imagine you need to display

`Welcome to JavaScript!` 10 times.

Without a function, you have to write

```
document.write("Welcome to JavaScript");
```

again and again.

Instead...

Create one function and call it whenever needed.

Definition

A **Function** is a reusable block of code that performs a specific task.

Remember

- Write Once
- Use Many Times

Real-Life Example

A TV Remote

Press the Power button anytime. You don't rebuild the TV every time. A function works the same way.

2 Defining a Function

What is Function Definition?

Defining a function means creating a function.

Syntax

```
function functionName()
{
    // code
}
```

Example

```
function greet()
{
    document.write("Welcome Students");
}
```

Visual

Function `greet()`

↓

Stores instructions

↓

Not executed yet

Key Point

Defining a function does **NOT** execute it.

3 Calling a Function

What is Function Calling?

Calling means executing the function.

Example

```
function greet()
{
    document.write("Welcome Students");
}

greet();   // function call
```

Output

```
Welcome Students
```

Flow Diagram

Program Starts

↓

`greet();`

↓

Function Executes

↓

Output Displayed

Remember

- Function definition stores the code.
- Function call runs the code.

Prepared by – *Prof. Priti Chavhan*

2.2 Functions in JavaScript

1 Writing a Function

Example

Find the square of a number.

```
function square()
{
    let num = 5;
    document.write(num*num);
}

square();   // function call
```

Output

25

Real-Life Example

Calculator

Press Square button

↓

Calculates automatically

2 Function with Arguments

Why Arguments?

Arguments allow us to pass different values.

Syntax

```
function functionName(parameter)
{
    // code
}
```

Example

```
function greet(name)
{
   document.write("Hello "+name);
}

greet("Rahul");  // argument passed
```

Output

Hello Rahul

Visual

Rahul

↓

name (Parameter)

↓

Hello Rahul (Output)

Key Point

- **Parameter** → Variable inside function
- **Argument** → Actual value passed

3 Multiple Arguments

Example

```
function add(a, b)
{
    document.write(a+b);
}

add(20, 30);  // arguments passed
```

Output

50

Visual

20

+

30

↓

50

Real-Life Example

Price + GST

or

Length + Width

2.2 Functions in JavaScript

Scope of Variables

What is Scope?

Scope means
Where a variable can be used.

Two Types

1. Local Variable
2. Global Variable

1 Local Variable

- ✓ Declared inside a function.
- ✓ Can be used only **inside that function.**

Accessible only inside the box

Example

```
function test()
{
    let x = 10;
    document.write(x);
}

test();    // function call
```

Output

```
10
```

Outside the function

```
x
```

✗ Error

Key Point
Local variables are temporary. They exist only inside the function.

2 Global Variable

- ✓ Declared outside a function.
- ✓ Can be used **anywhere.**

Accessible throughout the entire program

Example

```
let x = 10;
function test()
{
    document.write(x);
}

test();    // function call
```

Output

```
10
```

Key Point
Global variables remain available throughout the program.

Summary

Local Variable	Global Variable
• Declared inside a function	• Declared outside a function
• Used only inside that function	• Used anywhere in the program
• Temporary	• Available throughout

Tip
Use local variables whenever possible to keep your code safe and error-free.

Prepared by – *Prof. Priti Chavhan*

2.2 Functions in JavaScript

PREDICT THE OUTPUT

Read each program carefully and choose the correct output.

1 Program 1

```
function hello()
{
    document.write("Hello");
}

hello();
```

Output?

- A Hello
- B Error
- C Nothing

2 Program 2

```
function add(a,b)
{
    document.write(a+b);
}

add(10,20);
```

Output?

- A 30
- B 1020
- C Error

3 Program 3

```
let x=5;
function test()
{
    document.write(x);
}
test();
```

Output?

- A 5
- B Error
- C Undefined

Tip: Think about whether the function is called, whether values are passed, and where variables are declared.

2.3 Calling a Function

Think About It!

Imagine you have saved your friend's phone number.

Saving the number is **not enough.**

You must **dial the number** to talk to your friend.

Similarly,

- Defining a function **stores the instructions.**
- Calling a function **executes those instructions.**

Definition

A **Function Call** means executing a previously defined function.

- **Define** → Store the instructions
- **Call** → Execute the instructions

Real-Life Example

Mobile Contact

Save Contact → **Function Definition**

Call Contact → **Function Call**

Prepared by-*Prof. Priti Chavhan*

2.3 Calling a Function

1 Calling a Function Without Arguments

What is Calling Without Arguments?

The function is called without passing any values.

Syntax

```
function functionName()
{
    // code
}

functionName();
```

Example

```
function welcome()
{
   document.write("Welcome Students");
}

welcome();
```

Output

Welcome Students

Flow

→ `welcome();` → Function Executes → Output Displayed

Best used when the output is always the same.

2 Calling a Function With Arguments

Why Use Arguments?

Arguments allow different values to be passed every time.

Example

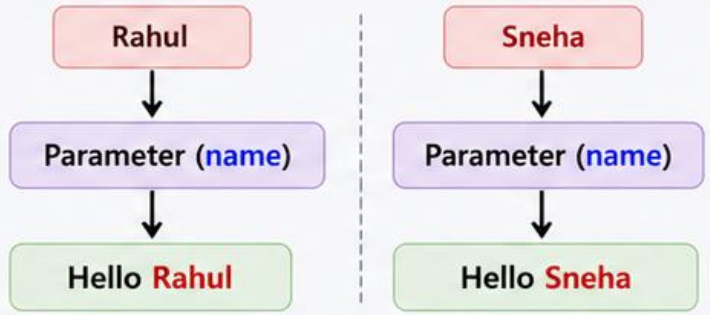
```
function greet(name)
{
    document.write("Hello " + name);
}

greet("Rahul");
greet("Sneha");
```

Output

Hello Rahul

Hello Sneha

Visual



Key Point

- Parameter → Variable inside function
- Argument → Actual value passed while calling

Prepared by– *Prof. Priti Chavhan*

2.3 Calling a Function

3 Calling Function from HTML

What is Calling from HTML?

A JavaScript function can be called when the user performs an action on a webpage.

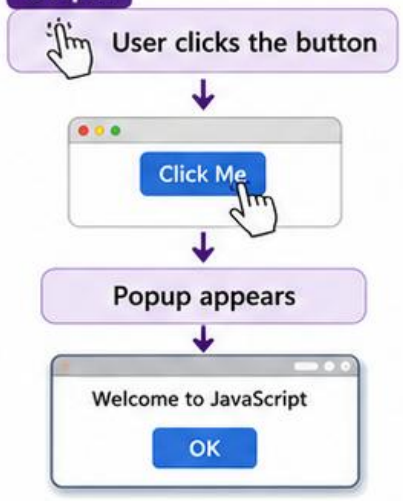
Examples

- Clicking a button
- Double-clicking
- Moving the mouse
- Pressing a key

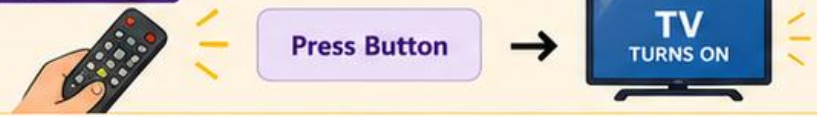
Example

```
<!DOCTYPE html>
<html>
<body>
   <button onclick="showMessage()">
       Click Me
   </button>
   <script>
   function showMessage()
   {
       alert("Welcome to JavaScript");
   }
   </script>
</body>
</html>
```

Output



Real-Life Example



4 Function Calling Another Function

What is Nested Function Calling?

One function can call another function.

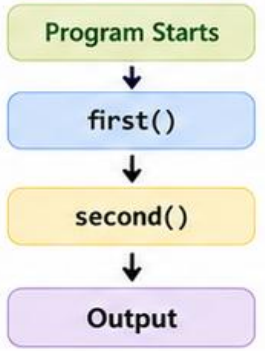
Output

First Function
Second Function

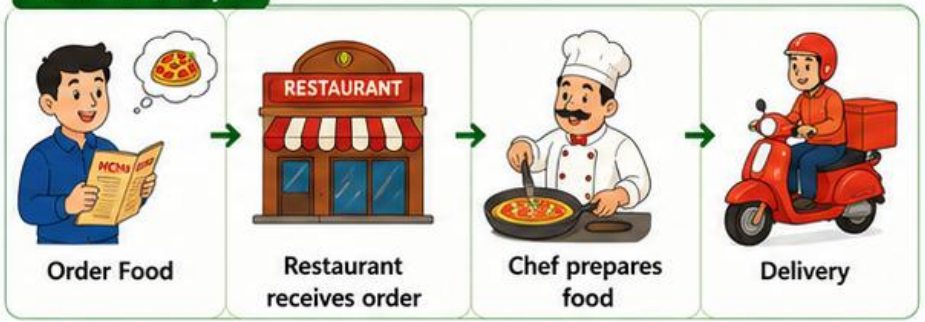
Example

```
function first()
{
   document.write("First Function<br>");
   second();
}

function second()
{
   document.write("Second Function");
}

first();
```

Flow Diagram



Real-Life Example



Each step is like **one function calling another.**

Prepared by– *Prof. Priti Chavhan*

2.3 Calling a Function

Function Without Return vs Function With Return

	Without return	With return
Output	Displays output directly	Sends value back
How it works	Uses document.write()	Uses **return**
Result	Cannot reuse result	Result can be stored in a variable
Flexibility	Simpler	More flexible

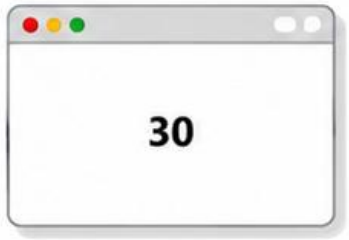
Remember: Use **return** when you want to use the result later in your program.

Example

Without return

```
function add()
{
   document.write(10+20);
}
```

Output displayed directly on the screen.

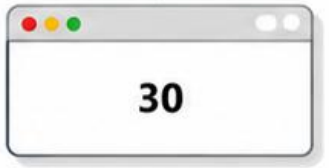


With return

```
function add()
{
    return 10+20;
}
```

Value is returned and can be stored in a variable.

```
let result = add();
document.write(result);
```



Prepared by- *Prof. Priti Chavhan*

2.3 Calling a Function

Predict the Output

Read each program carefully and choose the correct output.

Program 1

```
1  function hello()
2  {
3      document.write("Hello");
4  }
5
6  hello();
6
```

Output?

A. Hello
B. Error
C. Nothing

Program 2

```
1  function add(a,b)
2  {
3      return a+b;
4  }
5
6  document.write(add(10,20));
6
```

Output?

A. 30
B. 1020
C. Error

Program 3

```
1  function first()
2  {
3      document.write("A");
4      second();
5  }
6  function second()
7  {
9      document.write("B");
10 }
11 first();
```

Output?

A. AB
B. BA
C. Error

Think → Code → Run → Predict → Learn!

Prepared by– *Prof. Priti Chavhan*

Key Points…

- A function is a block of code that performs a specific task.
- Every C program must contain the main() function.
- Functions help divide large programs into smaller modules.
- Functions make programs easier to read, maintain and debug.
- Functions improve code reusability and efficiency.

4.2 Library Functions

The C Library is a collection of header files and library functions used to implement common operations such as input/output and string handling in the C programming language.

The C library is divided into different subsections. Each subsection has a header file that defines the objects found in that section of the library.

Standard Header Files:

<assert.h> <float.h> <math.h> <stdarg.h> <stdlib.h>

<ctype.h> <limits.h> <setjmp.h> <stddef.h> <string.h>

<errno.h> <locale.h> <signal.h> <stdio.h> <time.h>

The **Table 4.2.1** lists some commonly used **library functions with their descriptions and corresponding header files.**

4.2 Library Functions

Table: Commonly Used Library Functions

Function	Header File	Description
`int abs(int x)`	stdlib.h	Returns the absolute value of an integer argument x.
`double ceil(double x)`	math.h	Returns the smallest integer value greater than or equal to x.
`double cos(double x)`	math.h	Returns the cosine of a radian angle x.
`double cosh(double x)`	math.h	Returns the hyperbolic cosine of x. The value x is expressed in radians.
`double exp(double x)`	math.h	Returns the value of e raised to the xth power.
`double fabs(double x)`	math.h	Returns the absolute value of x (a negative value becomes positive, positive value is unchanged).
`double floor(double x)`	math.h	Returns the largest integer value less than or equal to x.

4.2 Library Functions

`double fmod(double x, double y)`	math.h	Returns the remainder of x divided by y.
`int getchar()`	stdio.h	Returns a character from standard input.
`double log10(double x)`	math.h	Returns the common logarithm (base-10 logarithm) of x.
`double pow(double x, double y)`	math.h	Returns the value of x to the power of y.
`printf()`	stdio.h	Sends formatted output to standard output.
`int putchar(int c)`	stdio.h	Sends the character given by c to standard output.
`int rand(void)`	stdlib.h	Returns a random positive integer.
`int scanf()`	stdio.h	Reads the formatted input from the standard input.
`double sin(double x)`	math.h	Returns the sine of a radian angle x.
`double sqrt(double x)`	math.h	Returns the square root of x.

4.2 Library Functions

`void srand(unsigned seed)`	stdlib.h	Initializes the random number generator.
`double tan(double x);`	math.h	Returns the tangent of a radian angle x.
`int toascii(int c);`	ctype.h	Converts value of c to ASCII.
`int tolower(int c);`	ctype.h	Converts the uppercase letter c to the corresponding lowercase letter.
`int toupper(int c);`	ctype.h	Converts the lowercase letter c to the corresponding uppercase letter.

4.2 Writing User Defined Functions

A user defined function is a function that is created by the programmer to perform a specific task in a program.

C language provides many library functions such as printf(), scanf(), sqrt(), etc., but sometimes programmers need to perform tasks that are not available in the standard library. In such cases, programmers create their own functions called user defined functions.

Definition

A User Defined Function (UDF) is a function written by the programmer to perform a specific operation in a program.

Example

A programmer may create functions like:

- add() → to add two numbers
- factorial() → to calculate factorial
- display() → to print output

4.2 Writing User Defined Functions

Components of User Defined Functions

A user defined function in C has three important parts:

1. Function Declaration (Prototype)
2. Function Definition
3. Function Call

These three parts work together for the proper execution of a function.

1. Function Declaration (Function Prototype): A function declaration tells the compiler about:

- Function name
- Return type
- Number of parameters
- Type of parameters

It informs the compiler before the function is used in the program.

4.2 Writing User Defined Functions

Syntax: return_type function_name(parameter_list);

Example

int add(int, int);

Explanation:

int → return type

add → function name

(int, int) → parameters

The semicolon ; indicates that this is only a declaration, not the function body.

Purpose of Function Declaration

1. It informs the compiler that a function exists.
2. It helps the compiler perform **type checking**.
3. It allows the function to be called **before its definition**.

4.2 Writing User Defined Functions

2. Function Definition

A function definition contains the actual code of the function that performs the required task.

Syntax

```
return_type function_name(parameter_list)
{
   statements;
}
```

Explanation

- int → function returns an integer value
- add → function name
- a and b → parameters
- sum → stores result
- return sum → sends result back to calling function

```
int add(int a, int b)
{
    int sum;
    sum = a + b;
    return sum;
}
```

4.2 Writing User Defined Functions

3. Function Call

A function call is used to execute the function in the program.

When a function is called, the program control transfers to that function, executes the statements, and then returns back to the calling function.

```
result = add(10, 20);
```

Syntax: function_name(arguments);

Explanation:

- add() function is called
- 10 and 20 are arguments
- Function calculates sum and returns value

General Form of a Function in C

In C programming, a function is a block of code that performs a specific task.
Every function follows a general structure that defines how the function is written and used.

The general form of a function shows the basic syntax and components of a function.

General Syntax of a Function:

```
return_type function_name(parameter_list)
{
    // variable declaration
    // executable statements
    return value;
}
```

Explanation of Each Part:

1. Return Type

The **return type** specifies the type of value that the function will return to the calling function.

General Form of a Function in C

2. Function Name

The function name is the identifier used to call the function.

Rules for function name:

1. Must start with a letter or underscore
2. Cannot contain spaces
3. Cannot be a keyword

Examples:
add()
display()
calculateSum()

3. Parameter List

Parameters are the inputs given to the function.
They are written inside parentheses ().

Example:
add(int a, int b)

Here:

- a and b are parameters
- They receive values when the function is called

If no parameters are required, we write:
function_name(void) or function_name()

General Form of a Function in C

4. Function Body

The function body is enclosed in curly braces { }.

It contains:
Variable declarations
Statements to perform the task

Example:

```
{
   int sum;
   sum = a + b;
   return sum;
}
```

5. Return Statement

The return statement is used to send the result back to the calling function.

Syntax: return value;

Example: return sum;

This sends the value of sum to the function that called it.

If the function type is void, then no value is returned.

Scope of Variables

The scope of a variable refers to the region of the program where the variable can be accessed.

In C programming, variables are mainly classified into:

1. Local Variables
2. Global Variables

1. Local Variables

Definition:

- ✓ A local variable is a variable that is declared inside a function or block.
- ✓ It can be accessed only within that function.

Characteristics of Local Variables:

- ➢ Declared inside a function
- ➢ Accessible only within that function
- ➢ Created when function starts
- ➢ Destroyed when function ends

Scope of Variables

Example of Local Variable

```c
#include<stdio.h>

void display()
{
    int a = 10;   // local variable
    printf("Value of a = %d", a);
}

int main()
{
    display();
    return 0;
}
```

Explanation:

- Variable a is declared inside display() function.
- It cannot be used outside this function.

Scope of Variables

2. Global Variables

Definition:

A global variable is a variable that is declared outside all functions.

It can be accessed by all functions in the program.

Characteristics of Global Variables

- ✓ Declared outside functions
- ✓ Accessible to all functions
- ✓ Exists throughout the program execution

Example of Global Variable:

```c
#include<stdio.h>

int a = 10;   // global variable

void display()
{
    printf("Value in display = %d\n", a);
}

int main()
{
    printf("Value in main = %d\n", a);
    display();

    return 0;
}
```

Difference Between Local and Global Variables

Feature	Local Variable	Global Variable
Declaration	Inside function	Outside all functions
Scope	Only inside function	Entire program
Lifetime	During function execution	Whole program execution
Accessibility	Limited	Available to all functions

4.4 Function Parameters in C

1. Introduction to Function Parameters

When we call a function, we often need to pass some data to it so that it can perform a task. These values are called parameters (or arguments).

Definition: Function parameters are the values passed to a function when it is called.

```c
#include<stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main()
{
    int result;

    result = add(10, 20);

    printf("Sum = %d", result);

    return 0;
}
```

Explanation:

- 10 and 20 → **Arguments (Actual Parameters)**
- a and b → **Formal Parameters**

Types of Parameters:

1. Actual Parameters (Arguments) → passed during function call
2. Formal Parameters → received in function definition

2. Parameter Passing Methods

In C, there are two ways to pass parameters:

1. Call by Value
2. Call by Reference

1. Call by Value

Definition: In call by value, a copy of the actual parameter is passed to the function.

The function works on the copy, not on the original variable.

Key Point

👉 Changes made inside the function do NOT affect the original variable.

Example:

```c
#include<stdio.h>

void change(int x)
{
    x = 50;
    printf("Inside function: %d\n", x);
}

int main()
{
    int a = 10;

    change(a);

    printf("Outside function: %d", a);

    return 0;
}
```

Output:

```
Inside function: 50
Outside function: 10
```

Explanation

- a = 10 is passed to function
- Copy of a is stored in x
- x changes to 50
- Original value a remains unchanged

2. Call by Reference

Definition

In call by reference, the address of the variable is passed to the function.

The function works on the original variable using pointers.

Key Point

👉 Changes made inside the function affect the original variable.

Explanation

- Address of a is passed using &a
- Pointer x stores address
- *x = 50 changes original value of a

```c
#include<stdio.h>

void change(int *x)
{
    *x = 50;
}

int main()
{
    int a = 10;

    change(&a);

    printf("Value of a = %d", a);

    return 0;
}
```

Output

```
Value of a = 50
```

Difference Between Call by Value and Call by Reference

Feature	Call by Value	Call by Reference
What is passed	Copy of value	Address of variable
Effect on original value	No change	Changes original value
Memory	Separate memory	Same memory
Use of pointers	Not used	Used

3. Function Return Values

Definition: A function can return a value to the calling function using the return statement.

```c
#include<stdio.h>

int square(int x)
{
    return x * x;
}

int main()
{
    int result;

    result = square(5);

    printf("Square = %d", result);

    return 0;
}
```

Explanation

- Function calculates square
- `return x * x` sends result back
- Value is stored in `result`

4. Function Return Types

Definition: The return type specifies the type of value a function returns.

Common Return Types

Return Type	Meaning
int	returns integer
float	returns decimal
char	returns character
double	returns large decimal
void	returns nothing

5. Declaring Function Return Types

When declaring a function, we must specify its return type.

Syntax:

```
return_type function_name(parameters);
```

Example:

```
int add(int, int);
float average(float, float);
void display();
```

Important Rule:

👉 If return type is not specified, C assumes it as int (older C standard).

👉 It is always a good practice to explicitly declare return type.

6. Return Statement

Definition:

The **return statement** is used to:

1. Return a value from a function
2. Exit from the function

Syntax:

```
return expression;
```

Example:

```
return sum;
```

```c
#include<stdio.h>

int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}

int main()
{
    int result;

    result = max(10, 20);

    printf("Maximum = %d", result);

    return 0;
}
```

4.5 Recursive Functions in C

1. Introduction to Recursion

A **recursive function** is a function that **calls itself** to solve a problem.

Instead of solving the entire problem at once, recursion breaks it into **smaller subproblems of the same type.**

Definition: A recursive function is a function that calls itself repeatedly until a certain condition is satisfied.

Simple Idea 👉 "Function calling itself again and again"

Real Life Example: Think of **standing between two mirrors**

You see **infinite reflections** of yourself.

Similarly, a recursive function keeps calling itself again and again.

Basic Structure of Recursion

Every recursive function must have two important parts:

1. Base Case (Stopping Condition)
   - Condition where recursion stops
   - Prevents infinite calling
2. Recursive Case
   - Function calls itself with smaller input

General Syntax:

```
return_type function_name(parameters)
{
    if (base_condition)
        return value;

    else
        return function_name(smaller_problem);
}
```

Example of Recursive Function

Example 1: Factorial Using Recursion

Mathematical Formula: $n! = n \times (n-1)!$

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1$

Output: 120

```c
#include<stdio.h>

int factorial(int n)
{
    if(n == 0)
        return 1;   // base case
    else
        return n * factorial(n - 1);  // recursive call
}

int main()
{
    int num = 5;

    printf("Factorial = %d", factorial(num));

    return 0;
}
```

Step-by-Step Working

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120
```

How Recursion Works (Call Stack Concept)

Each function call is stored in memory (stack).

Execution happens like:

```
factorial(3)
→ factorial(2)
→ factorial(1)
→ factorial(0)
```

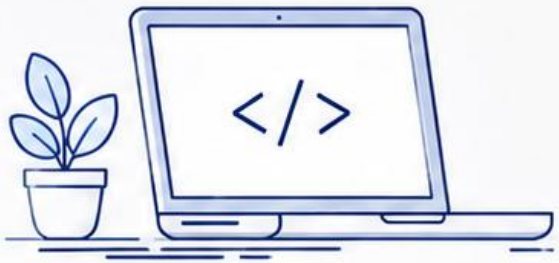
Then returns:

```
1 → 1 → 2 → 6
```

Thank You!

I hope this session has helped you to understand **basic concepts of Javascript** and its **basic usage.**

I hope this session has helped you understand Property Setters, Getters, and their practical applications in JavaScript.



Keep Learning, Keep Coding!

Prepared by – *Prof. Priti Chavhan*

SINCE 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU

॥ विश्वशान्तिर्ध्रुवं ध्रुवा ॥

Supported by UNESCO

UNESCO Chair for Human Rights, Democracy, Peace & Tolerance World Peace Centre (Alandi) Pune, India

Unit No. 3

Form and Event Handling

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Content:

3.1 Building blocks of a Form, properties and methods of form, button, text, text area, checkbox, radio button, select element.

3.2 Form events- mouse event, hey events.

3.3 Form objects and elements.

3.4 Changing attribute value dynamically.

3.5 Changing option list dynamically

3.6 Evaluating checkbox selection

3.7 Changing a label dynamically

3.8 Manipulating form elements

3.9 Intrinsic JavaScript functions, disabling elements, read only elements.

Course Outcome

- Create event-based web forms using Java script.

Learning Outcomes:

:

- Understand **HTML form elements** and their properties.
- Handle **mouse and keyboard events** using JavaScript.
- Access and **manipulate form elements dynamically**.
- Dynamically change **attributes, labels, options and checkbox selections**.
- Use common JavaScript functions and **enable, disable or make elements read-only**.

3.1 Building Blocks of a Form

What is a Form?

A form is an HTML element used to collect information from the user.

Examples:

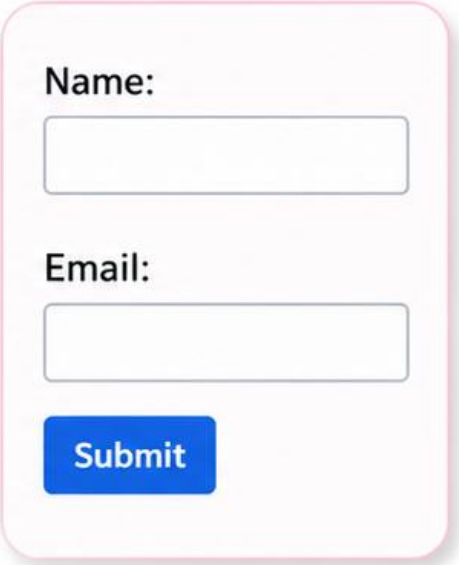
- Student registration form
- Login form
- Feedback form
- Admission form
- Online examination form

The main HTML tag used to create a form is:

```
<form> <!-- Form elements --> </form>
```

Basic Form Example

```
<form>
    <label>Name:</label>
    <input type="text">

    <br><br>
    <label>Email:</label>
    <input type="email">

    <br><br>
    <button type="submit">Submit</button>

</form>
```



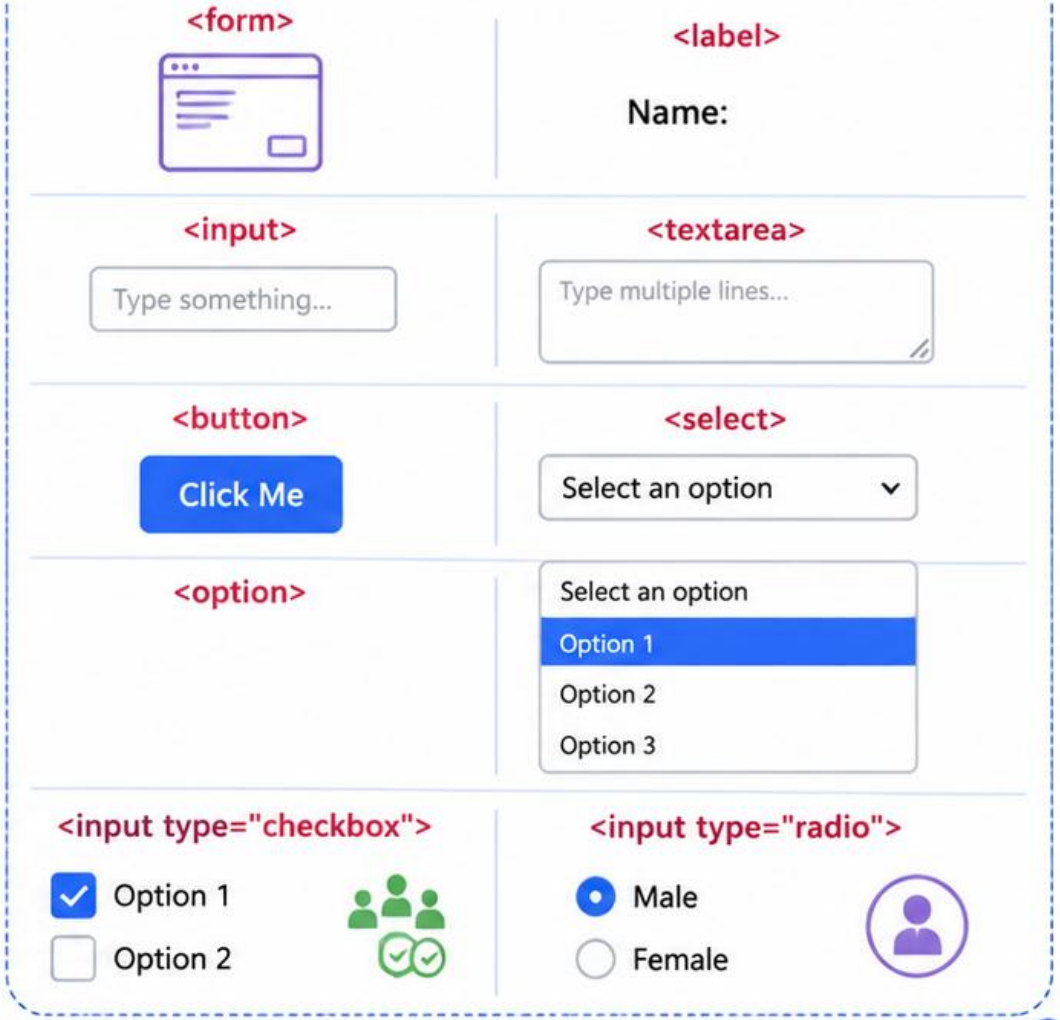
3.1 Building Blocks of a Form

Main Building Blocks of a Form

- Common form elements are:

Element	Purpose
`<form>`	Creates a form
`<label>`	Displays a label
`<input>`	Accepts different types of input
`<textarea>`	Accepts multiple lines of text
`<button>`	Creates a button
`<select>`	Creates a drop-down list
`<option>`	Defines an option in select
`<input type="checkbox">`	Allows multiple selections
`<input type="radio">`	Allows one selection from a group

Visual Representation



3.1 Building Blocks of a Form

1 Text Box

A text box is used to accept single-line text.

Syntax

```
<input type="text">
```

Example

```
<label>Student Name:</label>
<input type="text" id="studentName">
```

Important Attributes

Attribute	Purpose
id	Identifies the element
name	Gives a name to the form field
value	Sets the initial value
placeholder	Displays hint text
maxlength	Specifies maximum characters
readonly	Makes field read-only
disabled	Disables the field

Example

```
<input type="text"
       id="name"
       placeholder="Enter your name"
       maxlength="30">
```

Enter your name

2 Button

A button is used to perform an action.

Syntax

```
<button>Click Me</button>
```

Example

```
<button type="button"
     onclick="displayMessage()">
     Click Me
</button>
```

Click Me

JavaScript

```
function displayMessage() {
   alert("Button clicked!");
}
```

Common Button Types

```
<button type="button">Button</button>
<button type="submit">Submit</button>
<button type="reset">Reset</button>
```

Difference

Type	Purpose
button	Performs custom JavaScript action
submit	Submits the form
reset	Resets form fields

3 Text Area

A <textarea> is used to accept multiple lines of text.

Example

```
<label>Address:</label>
<textarea id="address"
          rows="5" cols="30">
</textarea>
```

It can be used for:

- Address
- Feedback
- Comments
- Description

Important Attributes

Attribute	Purpose
rows	Number of visible rows
cols	Number of visible columns
placeholder	Displays hint
readonly	Prevents editing
disabled	Disables the element

Example

```
<textarea
    rows="5"
    cols="30"
    placeholder="Enter your address">
</textarea>
```

Enter your address

4 Checkbox

A checkbox allows the user to select zero, one, or multiple options.

Syntax

```
<input type="checkbox">
```

Example

```
<label>Choose Subjects:</label>
<input type="checkbox" id="dbms">
<label>DBMS</label><br>
<input type="checkbox" id="javascript">
<label>JavaScript</label><br>
<input type="checkbox" id="python">
<label>Python</label>
```

Choose Subjects:
☐ DBMS ☐ JavaScript ☐ Python

The user can select multiple subjects.

Checking Checkbox

JavaScript provides the checked property.

```
let dbms = document.getElementById("dbms");

if (dbms.checked) {
   alert("DBMS is selected");
}
```

If DBMS is checked, the message will be displayed.

3.1 Building Blocks of a Form

5 Radio Button

Radio buttons are used when the user should select only one option from a group.

Example

```
<label>Gender:</label><br>
<input type="radio" name="gender" value="Male"> Male<br>
<input type="radio" name="gender" value="Female"> Female
```

Important: Radio buttons in the same group should have the **same name.**

Example

```
<input type="radio" name="course" value="CSE"> CSE<br>
<input type="radio" name="course" value="IT"> IT<br>
<input type="radio" name="course" value="AI"> AI
```

Only one course can be selected.

Same name = Same group
Only one option can be selected.

6 Select Element

The <select> element creates a drop-down list.

Example

```
<label>Select City:</label><br>
<select id="city">
  <option value="pune">Pune</option>
  <option value="mumbai">Mumbai</option>
  <option value="nashik">Nashik</option>
  <option value="nagpur">Nagpur</option>
</select>
```

Select City:

Pune

- Pune
- Mumbai
- Nashik
- Nagpur

Option

The <option> element defines an item in a drop-down list.

```
<option value="pune">Pune</option>
```

Form Properties

JavaScript provides properties to access and manipulate form elements.

Property	Purpose
value	Gets/sets value
checked	Checks checkbox/radio
disabled	Enables/disables element
readOnly	Makes element read-only
selectedIndex	Gets selected option index
options	Accesses options in select
length	Returns number of elements/options

Example

```
let name = document.getElementById("studentName");
console.log(name.value);
```

Form Methods

Some commonly used methods are:

1 submit()

Submits a form.

```
document.getElementById("myForm").submit();
```

2 reset()

Resets the form.

```
document.getElementById("myForm").reset();
```

3.2 Form Events

An **event** is an action that occurs in a web page.

Examples

- Mouse click
- Mouse movement
- Key press
- Form submission
- Change in input
- Focus on an element

JavaScript can **detect** these events and **execute** a **function.**

Mouse Events

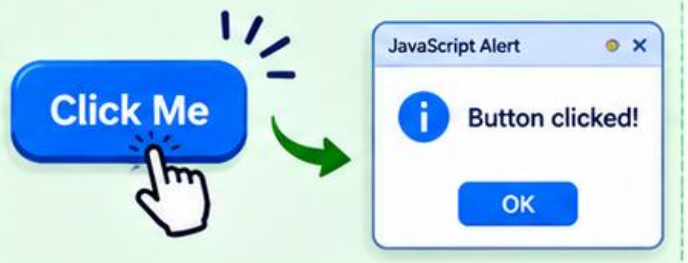
Mouse events occur when the user **interacts with the mouse.**

Common Mouse Events

Event	Description
onclick	Mouse button is clicked
ondblclick	Mouse button is double-clicked
onmouseover	Mouse moves over an element
onmouseout	Mouse leaves an element
onmousedown	Mouse button is pressed
onmouseup	Mouse button is released
onmousemove	Mouse moves

Events make web pages dynamic and interactive!

Example : onclick



HTML

```
<button onclick="showMessage()">
  Click Me
</button>
```

JavaScript

```
<script>
  function showMessage() {
    alert("Button clicked!");
  }
</script>
```

3.2 Form Events

Keyboard Events

Keyboard events occur when the user interacts with the keyboard.

Common Keyboard Events

Event	Description
onkeydown	Key is pressed down
onkeyup	Key is released
onkeypress	Key is pressed

Example

Type any key in the input box →

Open the browser console (**F12**) to see the message.

HTML

```
<input type="text" onkeydown="showMessage()">
```

JavaScript

```
<script>
  function showMessage() {
      console.log("Key pressed");
  }
</script>
```

Other Important Form Events

onchange

Occurs when the value of an element changes.

-- Select City --
Pune
Mumbai

```
<select onchange="showCity()" id="city">
  <option>Pune</option>
  <option>Mumbai</option>
</select>
```

```
function showCity() {
    alert("City changed");
}
```

onfocus

Occurs when an element receives focus.

Click or tab here

```
<input type="text" onfocus="focusMessage()">
```

```
function focusMessage() {
   console.log("Input selected");
}
```

onblur

Occurs when an element loses focus.

Click here and then click outside

```
<input type="text" onblur="blurMessage()">
```

```
function blurMessage() {
   console.log("Input lost focus");
}
```

Form events help us **respond** to user actions and make web pages **interactive.**

Keyboard Events | Mouse Events | Form Events

3.3 Form Objects and Elements

JavaScript treats HTML forms and their controls as **objects.**

For example:

```
<form id="studentForm">
   <input type="text" id="studentName">
   <button type="button">Submit</button>
</form>
```

Student Name:

Enter name | Submit

Key Point

Forms and their controls can be accessed and manipulated using JavaScript.

1 Accessing the Form

We can access the form using:

```
let form = document.getElementById("studentForm");
```

form

```
<form id="studentForm">
    ... form controls ...
</form>
```

Example:

```
console.log(form);
```

Output in Console

```
<form id="studentForm"> … </form>
```

The form is treated as an object in JavaScript.

2 Accessing Form Elements (Controls)

The most commonly used method is:

```
document.getElementById()
```

Example:

```
let studentName =
   document.getElementById("studentName");

console.log(studentName.value);
```

HTML Input

Student Name:

Priti

Output in Console

```
Priti
```

We can access any form control and read or change its properties like `value`, `type`, etc.

3 Using elements Collection

A form has an elements collection.

`form.elements` → Text Input | Button | Checkbox | etc.

Code Example

```
let form = document.getElementById("studentForm");
console.log(form.elements);
```

Output in Console

```
HTMLFormControlsCollection(2) [input#studentName, button, length: 2]
```

Example: Get Number of Elements

```
console.log(form.elements.length);
```

Output in Console

```
2
```

`form.elements` gives access to all the controls inside the form.

→ `length` property returns the total number of controls.

3.4 Changing Attribute Value Dynamically

JavaScript can change HTML attributes while the page is running.

We can change:

- value
- src
- href
- disabled
- readonly
- placeholder
- type

1 Changing Value

HTML:

```
<input type="text" id="name">
<button onclick="changeValue()">
    Change
</button>
```

JavaScript:

```
function changeValue() {
  document.getElementById("name").value = "Priti";
}
```

Output (Before Click)

Change

Output (After Click)

Priti

Change

The **value** attribute of the input field is changed to "Priti".

2 Changing Placeholder

```
document.getElementById("name")
    .placeholder = "Enter full name";
```

Before

Enter your name

After

Enter full name

The **placeholder** attribute is changed dynamically.

3 Changing Attribute Using setAttribute()

Syntax:

```
element.setAttribute("attribute", "value");
```

Example:

```
let input = document.getElementById("name");

input.setAttribute("placeholder", "Enter your name");
```

Before

Enter old name

After

Enter your name

setAttribute() is used to set or change any attribute.

4 Reading an Attribute

Use:

```
getAttribute()
```

Example:

```
let value = document.getElementById("name")
            .getAttribute("placeholder");
console.log(value);
```

Output in Console

Enter your name

getAttribute() is used to read the value of an attribute.

3.6 Evaluating Checkbox Selection

We can use the **checked** property to determine whether a checkbox is selected.

checked property returns:
- true → if the checkbox is selected
- false → if the checkbox is not selected

1 Single Checkbox Example

HTML

```
<input type="checkbox" id="html">
<label for="html">HTML</label>>
<br><br>
<button onclick="checkSelection()">
  Check
</button>
```

Live Preview

☐ HTML

Check

How it Works

If the checkbox is checked, alert shows "HTML is selected".

If not, alert shows "HTML is not selected".

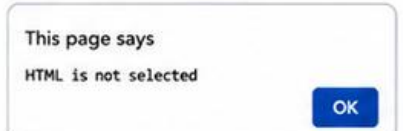
JavaScript

```
function checkSelection() {
  let html = document.getElementById("html");
  if (html.checked) {
     alert("HTML is selected");
  }
  else {
     alert("HTML is not selected");
  }
}
```

Output (Alert Box)

When checked = true

When checked = false



2 Multiple Checkbox Example

HTML

```
<input type="checkbox" id="dbms" >  <label for="dbms">DBMS</label><br>
<input type="checkbox" id="javascript">  <label for="javascript">JavaScript</label><br>
<input type="checkbox" id="python">  <label for="python">Python</label><br><br>
<button onclick="checkSubjects()">Check</button>
```

Live Preview

☐ DBMS
☐ JavaScript
☐ Python

Check

JavaScript

```
function checkSubjects() {
  let result = "";
  if (document.getElementById("dbms").checked) {
     result += "DBMS ";
  }
  if (document.getElementById("javascript").checked) {
     result += "JavaScript ";
  }
  if (document.getElementById("python").checked) {
     result += "Python ";
  }
  alert("Selected: " + result);
}
```

Example Outputs (Alert Box)

All selected →

DBMS & Python selected →

Only JavaScript selected →

None selected →

Key Point

Use the **checked** property to check if a checkbox is selected or not.

Tips

- Multiple checkboxs can be handled using the same logic.
- The result string collects all selected values.

Remember

- `html.checked` → `true` or `false`
- `alert()` → shows the result to the user

3.7 Changing a Label Dynamically

JavaScript can **change the text** displayed inside a `<label>`.

1 Changing Label Text

HTML

```
<label id="message">
   Enter your name
</label>
<input type="text">
```

Before

Enter your name

JavaScript

```
document.getElementById("message")
  .textContent = "Enter your full name";
```

After

Enter your full name

The label changes from "Enter your name" to "Enter your full name".

2 Dynamic Label Example

HTML

```
<label id="status">
   Status
</label><br><br>
<button onclick="changeLabel()">
  Change Label
</button>
```

Before Click

Status

Change Label

JavaScript

```
function changeLabel() {
   document.getElementById("status")
    .textContent = "Form Submitted";
}
```

After Click

Form Submitted

Change Label

Clicking the button changes the label text from "Status" to "Form Submitted".

3 textContent vs innerHTML

textContent	innerHTML
Used to change text.	Used to change HTML content.
`element.textContent = "Hello";`	`element.innerHTML = "<b>Hello</b>";`

Result in Browser

Using textContent	Using innerHTML
Hello	**Hello**

For simple label text, `textContent` is generally preferred.

Key Points

- Use `textContent` to change only the text inside a label.
- Use `innerHTML` when you want to add HTML elements (like `<b>`, `<i>`, `<span>`).
- `getElementById()` is used to access the label element.
- Changes are reflected immediately on the web page.

Prepared by-Prof. Priti Chavhan.

3.8 Manipulating Form Elements

Manipulation means changing or controlling a HTML element using JavaScript.

We can:

- Change value
- Change text
- Disable element
- Enable element
- Make read-only
- Change placeholder
- Change attributes
- Change options
- Show/hide

Example HTML

```
<label id="lblName">Enter your name:</label><br><br>
<input type="text" id="name" placeholder="Enter name"><br><br>
```

Set Value | Clear | Disable | Enable | Read-Only | Editable | Change Placeholder | Change Attribute

1 Changing Input Value

```
document.getElementById("name").value = "Priti";
```

Before: Enter name → After: Priti

2 Clearing Input

```
document.getElementById("name").value = "";
```

Before: Priti → After: Enter name

3 Disabling an Element

```
document.getElementById("name").disabled = true;
```

Before: Priti → After: Priti

4 Enabling an Element

```
document.getElementById("name").disabled = false;
```

Before: Priti → After: Priti

5 Making an Element Read-Only

```
document.getElementById("name").readOnly = true;
```

Before: Priti → After: Priti

6 Making It Editable Again

```
document.getElementById("name").readOnly = false;
```

Before: Priti → After: Priti

7 Changing Placeholder

```
document.getElementById("name").placeholder = "Enter full name";
```

Before: Enter name → After: Enter full name

8 Changing Attributes (using setAttribute)

```
document.getElementById("name").setAttribute("type","password");
```

Before: Priti → After: •••••

Key Takeaway

JavaScript gives full control over form elements. We can change their value, state, attributes and behavior dynamically.

Tip: Use `getElementById()` to access the element and then modify its properties.

Prepared by-Prof. Priti Chavhan.

3.9 Intrinsic JavaScript Functions

Intrinsic JavaScript functions are **built-in functions** provided by JavaScript that can be used to perform common operations.

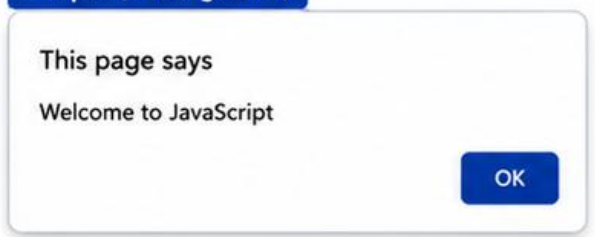
Function	Purpose
alert()	Displays message
confirm()	Displays confirmation
prompt()	Accepts input
parseInt()	Converts value to integer
parseFloat()	Converts value to decimal
isNaN()	Checks whether value is not a number
Number()	Converts value to number
String()	Converts value to string

1 alert()

Displays a message box.

```
alert("Welcome to JavaScript");
```

Output (Message Box)



2 confirm()

Asks the user for confirmation.

```
let result = confirm("Do you want
    to continue?");
console.log(result);
```

Output (Confirmation Box)

Returns

true

or

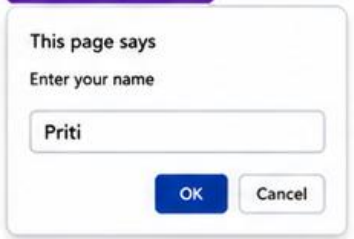
false

3 prompt()

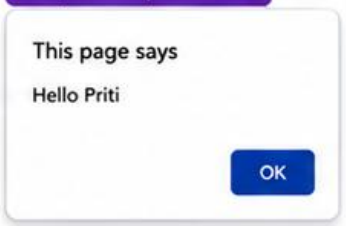
Accepts input from the user.

```
let name = prompt("Enter your name");
alert("Hello " + name);
```

Step 1: Input Box



Step 2: Output (Alert)



4 parseInt()

Converts a value into an integer.

```
let num = parseInt("25");
console.log(num);
```

Output (Console)

Explanation

The string "25" is converted to integer 25.

Tip: These intrinsic functions are part of JavaScript and are always available. They help in interacting with users and converting values easily.

3.9 Intrinsic JavaScript Functions

Intrinsic JavaScript functions are **built-in functions** provided by JavaScript that can be used to perform common operations and **control form element behavior**.

5 parseFloat()

Converts a value into a decimal number.

```
let price = parseFloat("25.50");
console.log(price);
```

Output (Console)

```
> 25.5
```

★ Decimal part is preserved.

6 isNaN()

Checks whether a value is Not a Number.

```
let value = "Hello";
console.log(isNaN(value));
```

Output (Console)

```
> true
```

★ "Hello" is not a number, so true.

7 Disabling Elements

An element can be disabled using the **disabled** property.

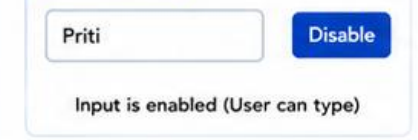
HTML

```
<input type="text" id="name">
<button onclick="disableInput()">
  Disable
</button>
```

JavaScript

```
function disableInput() {
  document.getElementById("name")
          .disabled = true;
}
```

Before Click



Input is enabled (User can type)

After Click

Input is disabled (User cannot type)

Disabled elements cannot be focused, clicked or edited by the user.

8 Enabling Elements

A disabled element can be enabled using the **disabled** property.

```
document.getElementById("name").disabled = false;
```

Before (Disabled)

Input is disabled

After (Enabled)

Input is enabled (User can type)

Setting **disabled = false** enables the element again.

9 Read-Only Elements

A **read-only** element can be viewed but its value cannot normally be edited by the user.

HTML Example

```
<input type="text" id="rollNo"
       value="101" readonly>
```

JavaScript

```
document.getElementById("rollNo")
  .readOnly = true;
```

Normal Input

User can edit this value.

Read-Only Input

User can view but cannot edit.

Read-only fields are useful for displaying fixed information.

Tip: Use these functions and properties to **interact with users** and **control form elements** effectively.

Quick Summary

`parseFloat()` → decimal | `isNaN()` → check Not a Number | `disabled` → disable/enable
`readonly` → view only

Prepared by-Prof. Priti Chavhan.

3.9 Intrinsic JavaScript Functions

These attributes control how users interact with form elements.

disabled vs readonly

This is an important **examination question.**

Feature	disabled	readonly
User Interaction	User cannot interact with the element.	User can focus/read the element.
Element Status	Element is disabled.	Element remains active.
Form Submission	Generally not submitted with form data.	Value can be submitted with form data.
Purpose	Used to prevent interaction.	Used to prevent editing.

Example

Using disabled

```
<input type="text"
       value="ABC"
       disabled>
```

ABC

Cannot type or focus

Using readonly

```
<input type="text"
       value="ABC"
       readonly>
```

ABC

Can focus and select text, but cannot edit

Key Takeaway

Use disabled when the user should not interact with the element at all.

Use readonly when the user should be able to view or focus, but not change the value.

Prepared by-Prof. Priti Chavhan.

Quick Revision Chart

FORM

Form Elements

- **Text**
  - Textarea
  - Checkbox
  - Radio
- **Button**
  - OK
- **Select**
  - Option

Events

- **Mouse**
  - click
  - mouseover
  - mouseout
- **Keyboard**
  - keydown
  - keyup
  - keypress

Forms **collect data** from users using different elements.
Events help us **respond to user actions**.

Tip: Form Elements are used to **take input**.
Events are used to **handle user interaction**.

Prepared by-Prof. Priti Chavhan.

Important Properties to Remember

Property	Example	Purpose
value	Priti `input.value`	Get or set the value of a text input, textaraa or other input elements.
checked	Subscribe `checkbox.checked`	Check whether a checkbox or radio button is selected or not.
disabled	Priti `input.disabled`	Disable or enable an element. When disabled, the user cannot interact with it.
readOnly	Priti `input.readOnly`	Make an element read-only. The user can see the value but cannot edit it.
selectedIndex	JavaScript `select.selectedIndex`	Get the index (position) of the currently selected option in a drop-down list.
options	JavaScript / JavaScript, Python, DBMS `select.options`	Access all the options in a drop-down list as a collection.
length	JavaScript `select.length`	Get the total number of options in a drop-down list.

Tip: These properties help us control and access form elements easily using JavaScript.
Remember: propertyName.property is the syntax!

Important Methods to Remember

	Method	Purpose	Example
1	getElementById()	Access an element using its id.	`document.getElementById("username");`
2	getElementsByName()	Access elements by their name.	`document.getElementsByName("gender");`
3	setAttribute()	Set the value of an attribute.	`element.setAttribute("placeholder", "Enter name");`
4	getAttribute()	Get the value of an attribute.	`element.getAttribute("class");`
5	removeAttribute()	Remove an attribute.	`element.removeAttribute("disabled");`
6	add()	Add option to a select element.	`select.add(option, index);`
7	remove()	Remove option from a select element.	`select.remove(index);`
8	submit()	Submit the form programmatically.	`document.forms["myForm"].submit();`
9	reset()	Reset all form elements.	`document.forms["myForm"].reset();`
10	createElement()	Create a new HTML element.	`var newDiv = document.createElement("div");`

Tip: These methods help us **access, modify, and manage** HTML elements and forms efficiently using JavaScript.

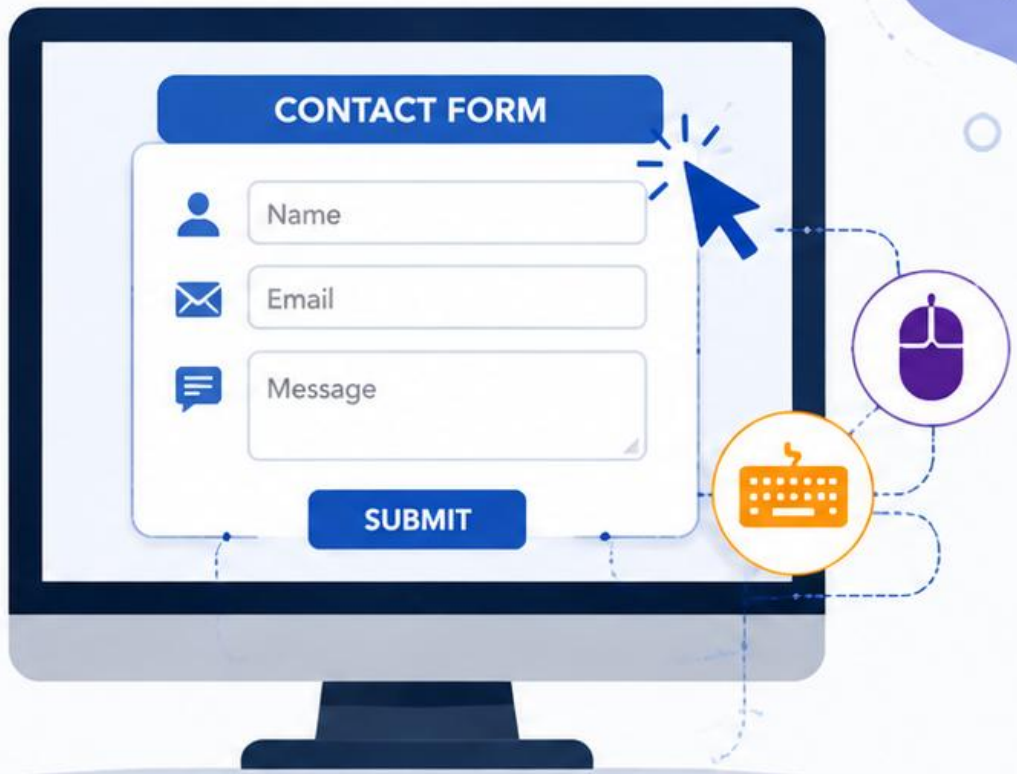
Remember: Understanding these methods is essential for **DOM manipulation** and **form handling**.

Prepared by-Prof. Priti Chavhan.

Thank You!

for learning

Form and Event Handling



Forms **collect data** and **Events** make web pages **interactive**.

Collect Data | Handle Events | Validate Input | Make Web Interactive

Prepared by-Prof. Priti Chavhan.

Unit No. 4

Cookies and Browser Data

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Course Outcome

- CO4 - Use JavaScript for handling cookies.

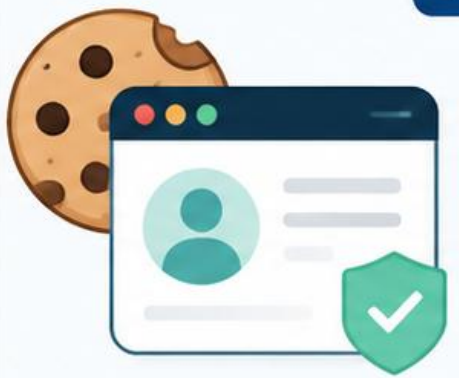
Content:

4.1 Cookies - basic of cookies, reading a cookie value, writing a cookie value, creating a cookie, deleting a cookie, setting the expiration crate of cookie

4.2 Browser — opening a window, giving the new window focus, window position, changing the content of window, closing a window, scrolling a, web page, multiple windows at once, creating a web page in new window, JavaScript in URLs, JavaScript security, Timer s, Browser location and history.

4.1 Cookies

What is a Cookie?



A **cookie** is a small piece of information stored by a website in the user's web browser.

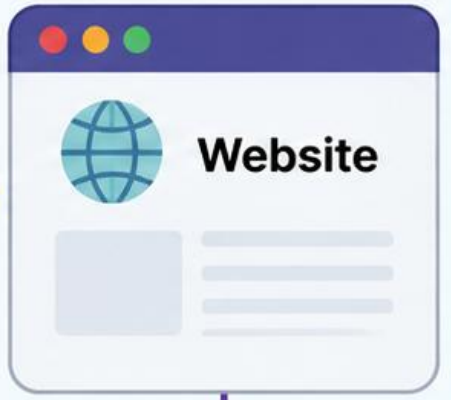
Cookies are used to remember information such as:

- Username
- User preferences
- Login information
- Language selection
- Website settings

Example

Suppose a student visits a website and enters:

Name = "Rahul"



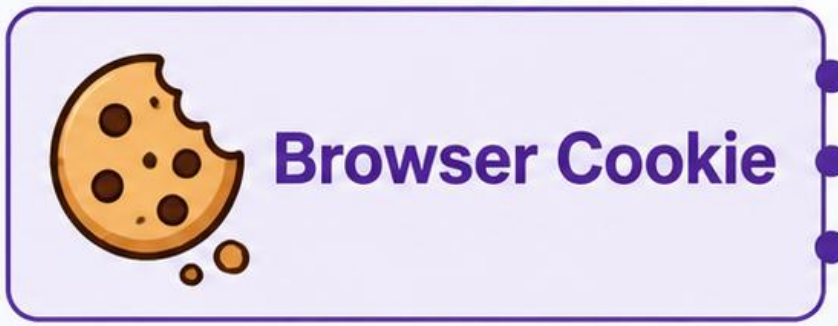
Cookie Stores
Name = "Rahul"

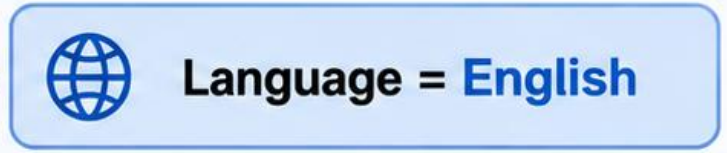
When the student visits the website again, the website can read the stored cookie value.

4.1 Cookies

Simple Representation

Stores Information





4.1 Cookies

Creating a Cookie

In JavaScript, cookies are created using the **document.cookie** property.

Syntax

```
document.cookie = "cookieName=cookieValue";
```

Example

```
document.cookie = "username=Rahul";
```

This creates a cookie named **username** with the value **Rahul**.

Writing a Cookie

Writing a cookie means storing information in the browser.

Example

```
document.cookie = "city=Pune";
```

Here:

- Cookie name → **city**
- Cookie value → **Pune**

Reading a Cookie

Cookies can be read using:

```
document.cookie
```

Example

```
alert(document.cookie);
```

If the browser contains:

```
username=Rahul
city=Pune
```

↓

the cookie information can be displayed.

Creating a Cookie with Expiry Date

By default, some cookies may not remain permanently stored.
We can specify an **expiry date.**

Example

```
document.cookie =
"username=Rahul; expires=Fri, 31 Dec 2027 12:00:00 UTC";
```

The cookie will remain available until the specified expiry date, subject to browser settings.

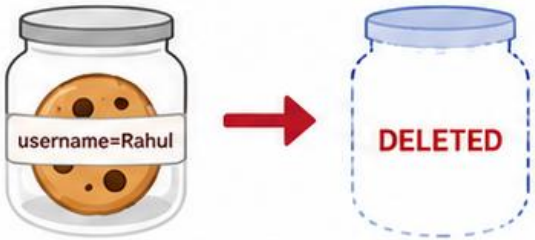
4.1 Cookies

Deleting a Cookie

A cookie can be deleted by setting its **expiry date** to a date in the past.

Example

```
document.cookie =
"username=Rahul;
expires=Thu, 01 Jan 1970
00:00:00 UTC";
```



Setting the expiry date in the past removes the cookie from the browser.

Simple Program: Create and Read a Cookie

```
<!DOCTYPE html>
<html>
<head>
    <title>Cookie Example</title>
    <script>
      function createCookie() {
         document.cookie = "username=Rahul";
         alert("Cookie Created Successfully");
      }
      function readCookie() {
         document.getElementById("output").innerHTML =
            document.cookie;
      }
   </script>
</head>
<body>
    <h2>JavaScript Cookie Example</h2>
    <button onclick="createCookie()">Create Cookie</button>
    <button onclick="readCookie()">Read Cookie</button>
    <p id="output"></p>
</body>
</html>
```

Output

JavaScript Cookie Example

Create Cookie | Read Cookie

username=Rahul

How it works?

1. Click on "**Create Cookie**" → A cookie named **username** is created.
2. Click on "**Read Cookie**" → The cookie value is displayed on the page.

4.1 Cookies

Cookies are **small pieces of data** stored in the user's browser to remember information about the user.

Types of Cookies

1 Session Cookie

- A session cookie is generally associated with the **current browser session**.
- It is commonly created **without specifying** an **expiry date**.

Example

```
document.cookie = "username=Rahul";
```

2 Persistent Cookie

- A persistent cookie is created with an **expiry date** so that it can remain available **beyond the current session**, depending on browser settings.

Example

```
document.cookie =
"username=Rahul; expires=Fri, 31 Dec 2027 12:00:00 UTC";
```

Note: Session cookies are **temporary**, while **persistent** cookies stay until they expire or are deleted by the user.

4.2 Browser Window

What is the Browser Window?

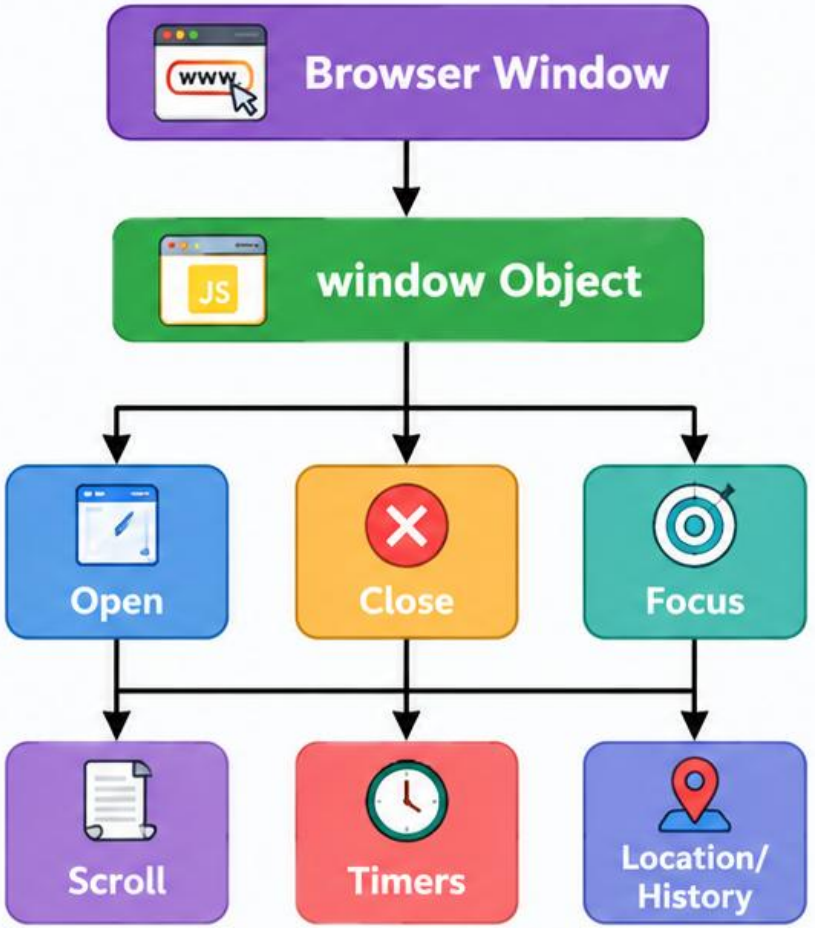
When we open a website in a browser such as Chrome, Edge, or Firefox, the webpage is displayed inside a browser window.

JavaScript provides the window object to interact with the browser window.

The window object represents the browser window and allows us to perform many operations related to it.

The window object is the global object in the browser. It is automatically available in JavaScript.

Simple Representation



The window object can be used to:

- Open a new window
- Close a window
- Give focus to a window
- Change window content
- Set window position
- Scroll a webpage
- Open multiple windows
- Use timers
- Work with browser location
- Work with browser history

4.2 Browser Window

Opening a New Window

JavaScript provides the **window.open()** method to open a new browser window or tab.

Syntax

```
window.open(URL, name, features);
```

Parameters

Parameter	Meaning
URL	Address of the webpage to open
name	Name or target of the window
features	Window properties such as width and height

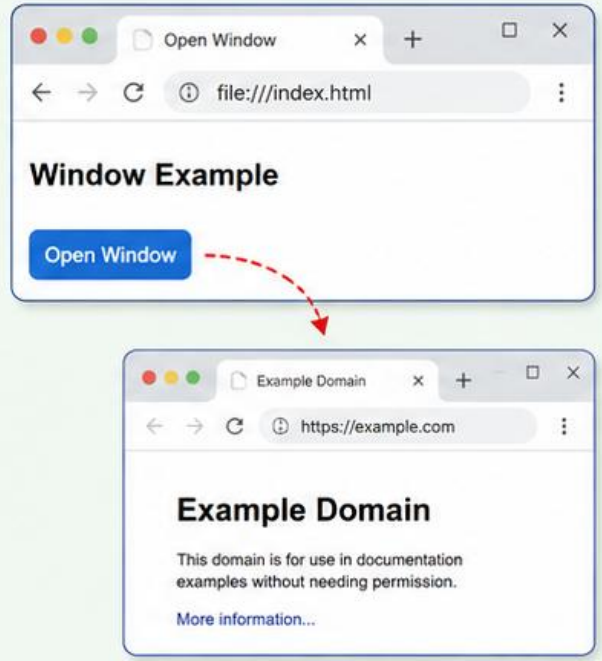
Example

```
window.open("https://example.com");
```

When this statement executes, the browser attempts to open the specified webpage.

Example Using a Button

```
<!DOCTYPE html>
<html>
<head>
   <title>Open Window</title>

   <script>
      function openWindow() {
         window.open("https://example.com");
      }
   </script>
</head>
<body>

   <h2>Window Example</h2>

   <button onclick="openWindow()">
      Open Window
   </button>

</body>
</html>
```



Working

Note:

Modern browsers may block pop-up windows depending on browser settings, especially when the window is not opened directly through a user action.

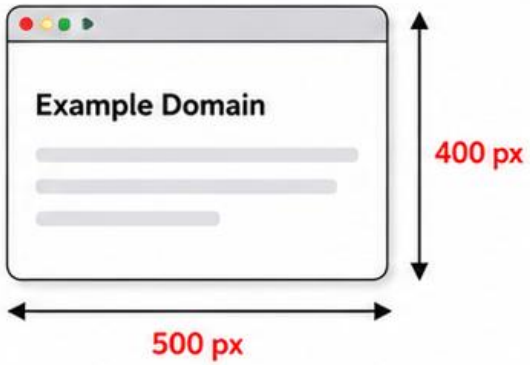
4.2 Browser Window

1 Opening a Window with a Specific Size

We can specify the `width` and `height` while opening a window.

Example

```
window.open(
  "https://example.com",
  "_blank",
  "width=500,height=400"
);
```



Meaning

`width = 500` ⇓ Width of the window = 500 pixels

`height = 400` ⇓ Height of the window = 400 pixels

Parameters

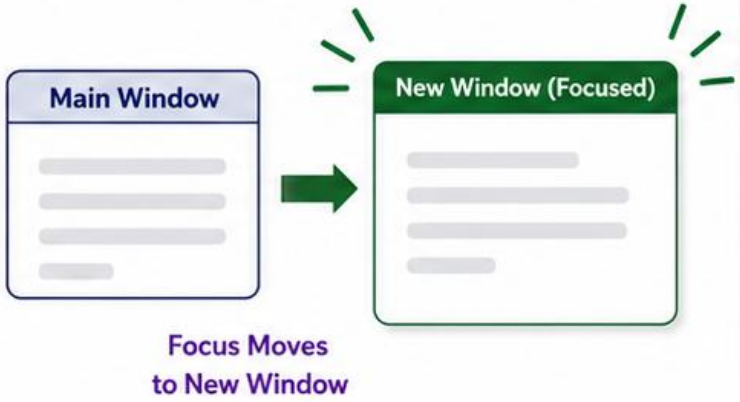
Property	Purpose
`width=500`	Sets the width of the window
`height=400`	Sets the height of the window

2 Giving Focus to a New Window

The `focus()` method gives focus to an already opened window.

Example

```
let newWindow = window.open(
  "https://example.com"
);

newWindow.focus();
```



Explanation

1. `window.open()` → Opens a new window
2. Window reference stored in `newWindow` → Keeps a reference of the opened window
3. `newWindow.focus()` → Gives focus to the opened window

The variable `newWindow` stores a reference to the newly opened window.

Then `newWindow.focus();` brings focus to that window.

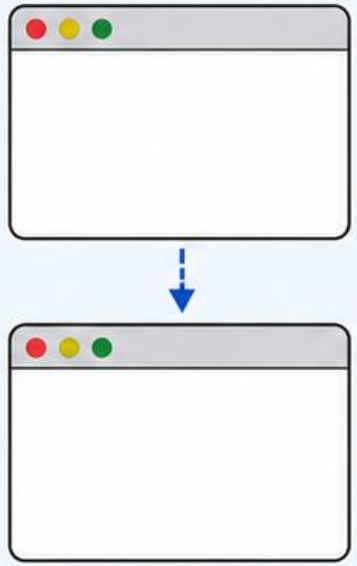
4.2 Browser Window

Closing a Window

The close() method is used to close a browser window.

Example

```
let newWindow;
function openWindow() {
    newWindow = window.open(
        "",
        "",
        "width=400,height=300"
    );
}
function closeWindow() {
    newWindow.close();
}
```



Working

1	**Click Open**	User clicks the Open button.
2	**A new window is created**	A new browser window is opened.
3	**Reference stored in newWindow**	The reference of the new window is stored in newWindow.
4	**Click Close**	User clicks the Close button.
5	**newWindow.close() executes**	The close() method executes.
6	**The opened window is closed**	The opened window is closed.

Important Point

Usually, JavaScript is permitted to close windows that were opened by the script itself.
Browser security restrictions may prevent a script from closing certain browser windows.

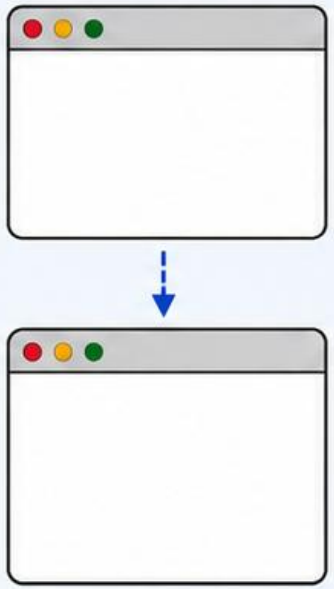
4.2 Browser Window

Closing a Window

The **close()** method is used to close a browser window.

Example

```
let newWindow;
function openWindow() {
    newWindow = window.open(
        "",
        "",
        "width=400,height=300"
    );
}
function closeWindow() {
    newWindow.close();
}
```



Working

1	**Click Open**	User clicks the Open button.
2	**A new window is created**	A new browser window is opened.
3	**Reference stored in newWindow**	The reference of the new window is stored in newWindow.
4	**Click Close**	User clicks the Close button.
5	**newWindow.close() executes**	The close() method executes.
6	**The opened window is closed**	The opened window is closed.

Important Point

Usually, JavaScript is permitted to close windows that were opened by the script itself.
Browser security restrictions may prevent a script from closing certain browser windows.

4.2 Browser Window

Changing the Content of a Window

JavaScript can write content into a newly opened window.

Example

```
let newWindow = window.open();

newWindow.document.write(
  "<h1>Welcome to JavaScript</h1>"
);
```

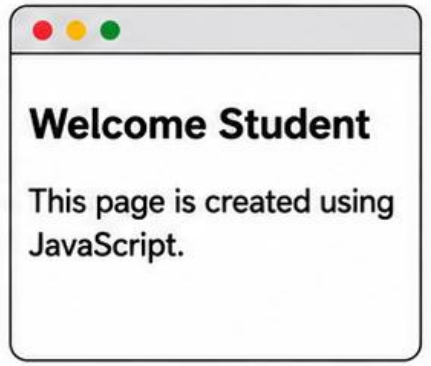
Explanation

1. `let newWindow = window.open();`
   → This opens a new window.

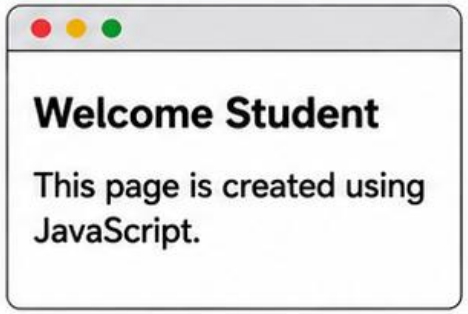
2. `newWindow.document.write( "<h1>Welcome to JavaScript</h1>" );`
   → writes HTML content into that window.

Another Example

```
function createPage() {
 let newWindow = window.open();

 newWindow.document.write(
    "<h1>Welcome Student</h1>"
 );

 newWindow.document.write(
   "<p>This page is created using
      JavaScript.</p>"
 );
}
```



Output



Important Point

Usually, JavaScript is permitted to close windows that were opened by the script itself. Browser security restrictions may prevent a script from closing certain browser windows.

4.2 Browser Window

Scrolling a Web Page

MIT-WPU
|| विश्वशांति वृष्टिः ||
MIT WORLD PEACE UNIVERSITY

Scrolling a Web Page

The scrollTo() method is used to scroll a webpage to a specified position.

Syntax

```
window.scrollTo(x, y);
```

Where:

- x represents the horizontal position.
- y represents the vertical position.

Example

```
window.scrollTo(0, 500);
```

Meaning

x = 0 → No horizontal movement

y = 500 → Move vertically to position 500

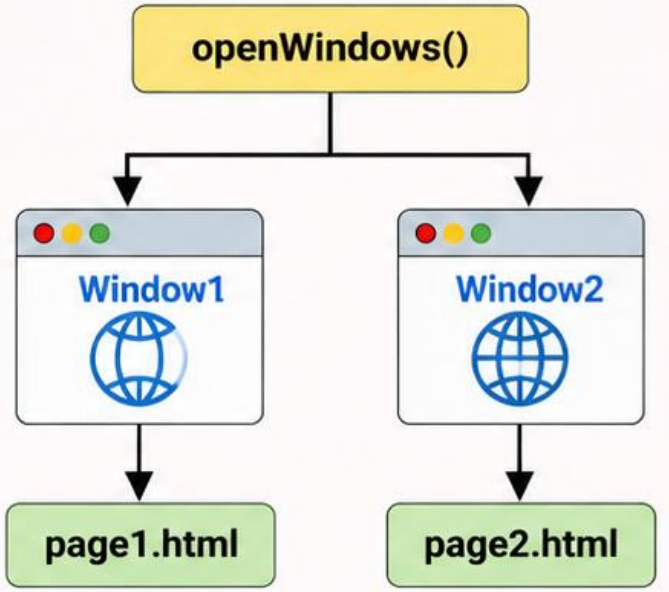
Multiple Windows

JavaScript can open more than one window.

Example

```
function openWindows() {

  window.open(
    "page1.html",
    "Window1"
  );

  window.open(
    "page2.html",
    "Window2"
  );
}
```

Working



This example opens two different webpages.

4.2 Browser Window

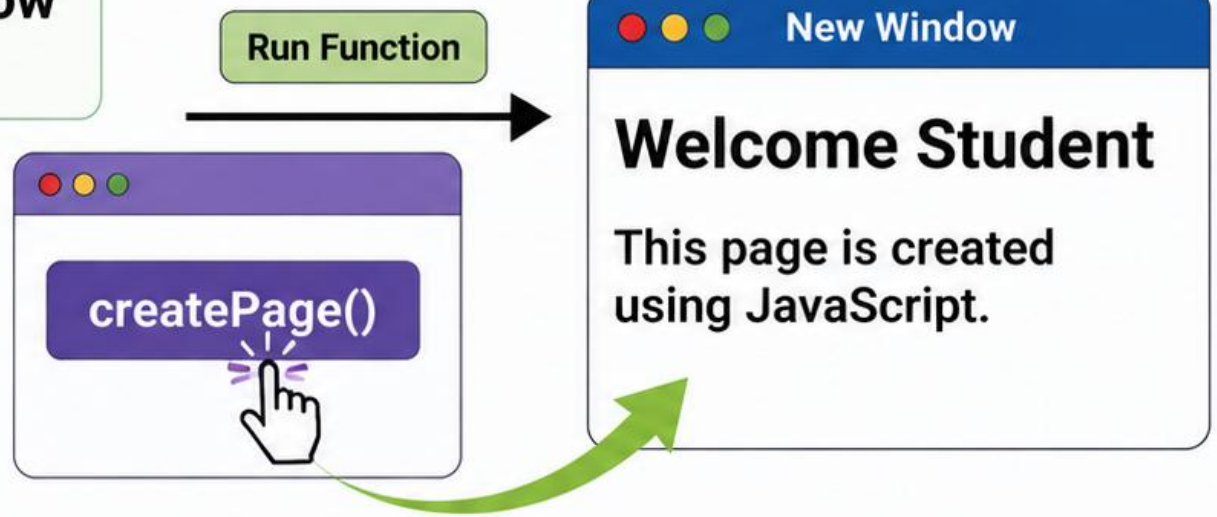
Creating a Web Page in a New Window

MIT-WPU
|| विश्वशांति वृंद ||
MIT WORLD PEACE UNIVERSITY

JavaScript can create a new browser window and write HTML content into it.

Example

```
function createPage() {
  let newWindow = window.open();

  newWindow.document.write(
    "<h1>Welcome Student</h1>"
  );

  newWindow.document.write(
    "<p>This page is created using JavaScript.</p>"
  );
}
```



Steps

1. `window.open()` opens a new window.
2. The reference is stored in `newWindow`.
3. `document.write()` writes HTML content into the new window.
4. The browser displays the newly created content.

4.2 Browser Window

JavaScript in URLs

MIT-WPU
MIT WORLD PEACE UNIVERSITY

JavaScript can be associated with a URL using the **javascript:** scheme.

Example

```
<a href="javascript:alert('Welcome')">
    Click Here
</a>
```

When the user clicks **Click Here**, the following statement executes:

```
alert('Welcome')
```

→ Therefore, an alert box appears.

For modern webpage development, **event handlers** or **JavaScript event listeners** are generally preferred over placing **JavaScript directly in URLs**.

Simple Flow

User clicks link

↓

JavaScript code executes

↓

`alert("Welcome")`

↓

Message box displayed

4.2 Browser Window

JavaScript Security

MIT-WPU
MIT WORLD PEACE UNIVERSITY

Modern browsers apply various security restrictions to JavaScript. These restrictions help protect users and their data.

1 Popup Windows May Be Blocked

Browsers may block popup windows.

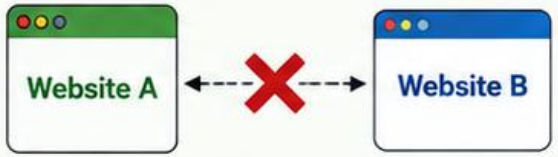
For example:

```
window.open("page.html");
```

may be blocked if it is not associated with a direct user action.

2 Restricted Access Between Different Websites

A script may be restricted from accessing from another origin.



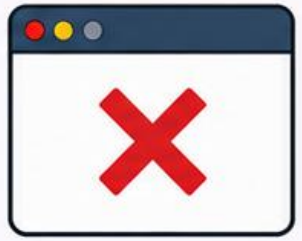
For example, a script running on one website cannot freely access the contents of every other website.

Some Important Points

3 Restrictions on Closing Windows

A script may not be allowed to close every browser window.

Typically, a script is more likely to be allowed to close a window that it opened itself.



4 Browser Settings Affect JavaScript

The behavior of some JavaScript features may depend on:

- Browser settings
- Security policies
- Popup blocking
- User permissions

Therefore, programs involving browser windows may behave differently in different browsers.

4.2 Browser Window

Timer

MIT-WPU
|| विश्वशांति श्रीमत् ||
MIT WORLD PEACE UNIVERSITY

JavaScript provides timers to execute code:

- After a specified time.
- Repeatedly after a fixed interval.

The syllabus includes Timers as part of browser-related features.

1 setTimeout()

The `setTimeout()` method executes a function once after a specified delay.

Syntax

```
setTimeout(functionName,
    milliseconds);
```

Example

```
function showMessage() {
    alert("Welcome!");
}

setTimeout(showMessage, 3000);
```

Explanation

showMessage
↓
Wait for 3000 milliseconds
↓
3000 milliseconds = 3 seconds
↓
Function executes once
↓
"Welcome!" is displayed

2 clearTimeout()

The `clearTimeout()` method is used to cancel a timeout before it executes.

Example

```
let timer = setTimeout(
    showMessage,
    5000
);

clearTimeout(timer);
```

Working

setTimeout()
↓
Timer is created
↓
Timer ID stored in variable timer
↓
clearTimeout(timer)
↓
Scheduled function is cancelled

4.2 Browser Window

Timer

1 setInterval()

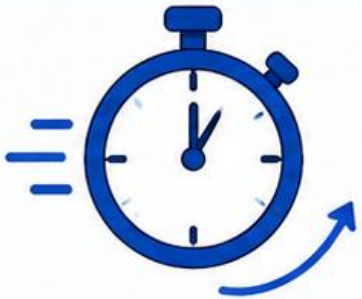
The `setInterval()` method repeatedly executes a function after a specified time interval.

Example

```
setInterval(showTime, 1000);
```

Meaning

1000 milliseconds = 1 second



Therefore:

showTime()

↓

After 1 second → Execute

↓

After 1 second → Execute again

↓

After 1 second → Execute again

↓

Continues repeatedly

2 clearInterval()

The `clearInterval()` method stops a repeating interval.

Example

```
let interval  = setInterval(
    showTime,
    1000
);

clearInterval(interval);
```

Explanation

- The ID returned by `setInterval()` is stored in the variable `interval`.
- `clearInterval(interval);` stops the repeated execution.

setInterval() repeats the task. **clearInterval()** stops it.

prepared by-Prof. Priti Chavhan

4.2 Browser Window

Difference Between setTimeout() and setInterval()

setTimeout()	VS	setInterval()
Executes once		Executes repeatedly
Runs after a specified delay		Runs after every specified interval
Can be stopped using clearTimeout()		Can be stopped using clearInterval()

Example

```
setTimeout(showMessage, 3000);
```

➡ Wait 3 seconds → Execute once

Example

```
setInterval(showTime, 1000);
```

➡ Every 1 second → Execute repeatedly

In Short: Use `setTimeout()` for one-time delay and `setInterval()` for repeated actions.

★ **Remember:** setTimeout() = Once, setInterval() = Repeat!

prepared by-Prof. Priti Chavhan

4.2 Browser Window

Browser History

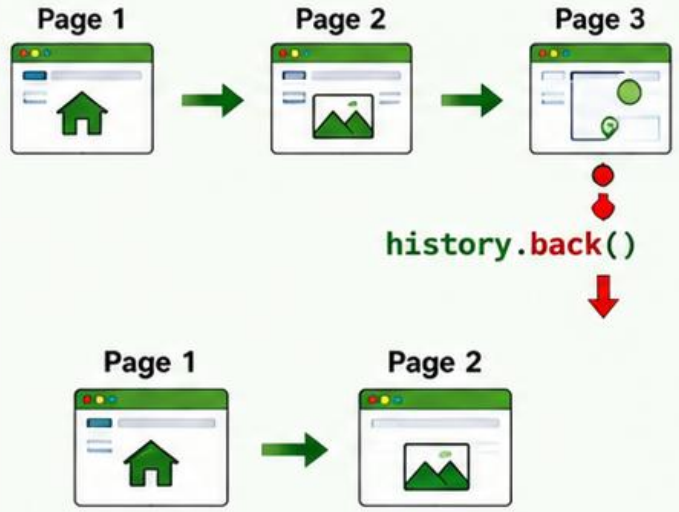
The history object provides access to the browser's session history.
It can be used to move backward or forward through pages visited in the current browser session.

1 history.back()

Used to move to the previous page.

```
history.back();
```

Example

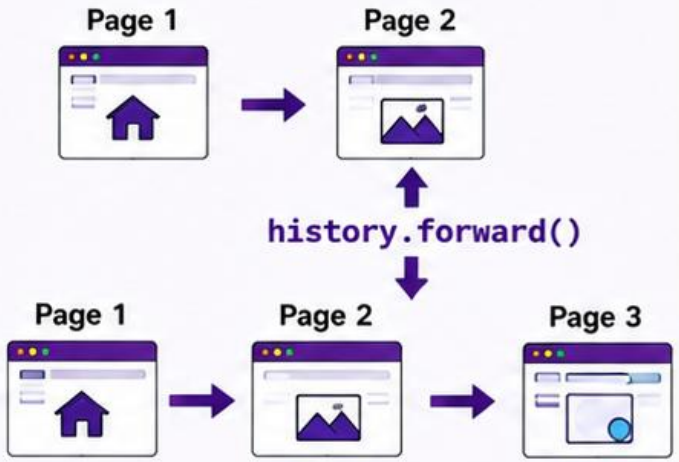


2 history.forward()

Used to move to the next page in the session history when available.

```
history.forward();
```

Example



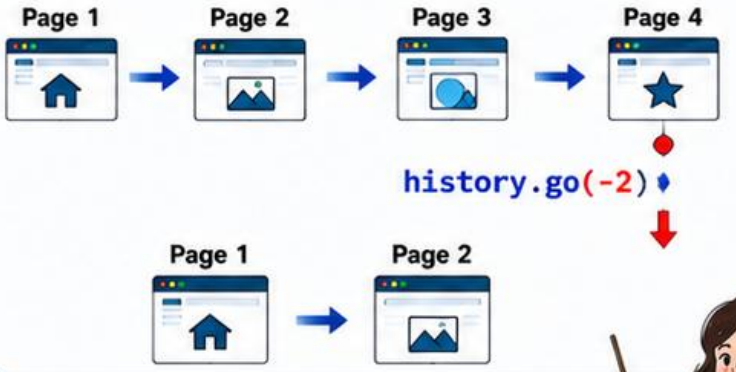
3 history.go()

The history.go() method moves through the browser history by a specified number of positions.

Example

```
history.go(-2);
```

This requests navigation two entries backward in the browser's session history.



Quick Tip: Use back(), forward(), and go() to give a better navigation experience!

Prepared by-Prof. Priti Chavhan

Thank You!

I hope this session has helped you to understand **basic concepts of Javascript** and its **basic usage.**

Keep Learning, Keep Coding!

Prepared by – *Prof. Priti Chavhan*

Unit No. 5

Regular Expression, Rollover and Frames

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Course Outcome

- CO5 - Create interactive webpage using regular expressions for validations.

Content:

5.1 Regular Expression -language of regular expression, finding nonmatching characters, entering a range of characters, matching digits and non-digits, matching punctuations and symbols, matching words. replacing the text using regular expressions, returning the matched regular expression object properties.

5.2 Frames — create a frame, invisible borders of frame, calling a child windows, changing a content and focus of a child window, writing to a child window, accessing elements of another child window.

5.3 Rollover - creating rollover, text rollover, Multiple actions for rollover, more efficient rollover.

5.1 Regular Expression

Introduction to Regular Expression

Regular Expression (Regex)

A Regular Expression is a pattern used to **search**, **match**, and **manipulate** text.

Example:

```
let pattern = /hello/;
```

This pattern searches for the word **hello**.

It is commonly used for:

- Searching specific text
- Validating user input
- Finding numbers or characters
- Replacing text
- Extracting required information

Key Point

Regular Expression describes a **pattern**, **not** a particular **text**.

5.1 Regular Expression(Continue..)

Creating a Regular Expression

A regular expression can be created in two ways.

1 Regular Expression Literal

```
let pattern = /hello/;
```

2 RegExp Object

```
let pattern = new RegExp("hello");
```

Example

```
let text = "Hello World";
let pattern = /Hello/;
console.log(pattern.test(text));
```

Output:

true

Important Method

test() checks whether the pattern is present in the given text.

Language of Regular Expression

A regular expression consists of:

Component	Meaning	Example
Literal characters	Match exact characters	/cat/
Character class	Match one character from a set	/[abc]/
Range	Match characters within a range	/[a-z]/
Digit	Match a digit	/\d/
Word	Match word character	/\w/
Quantifier	Specify repetition	/a+/
Anchor	Specify position	/^A/

Example

```
let pattern = /cat/;
```

Matches: ✔ cat

Does not match: ✘ dog

5.1 Regular Expression(Continue..)

Character Classes

What is a Character Class?

- A fcharacter class is used to match one character from a specified set.

Syntax: `[characters]`

Example:

`/[abc]/`

Matches:

- a
- b
- c

Matching a Set (Example)

```
let pattern = /[aeiou]/;
```

- This matches any vowel.

Example:

```
let text = "JavaScript";
console.log(/[aeiou]/.test(text));
```

Output: `true`

Finding Non-Matching Characters

- The ^ symbol inside a character class means NOT.

Syntax: `[^characters]`

Example:

```
let pattern = /[^0-9]/;
```

It can match:

• A • b • @ • #

But it does not match:

• 0 • 1 • 2 • 3
• 4 • 5 • 6 • 7
• 8 • 9

Remember

`[0-9]`	→	digit
`[^0-9]`	→	non-digit

Entering a Range of Characters

- A hyphen - is used to specify a range.

Common Ranges

Pattern	Meaning	Pattern	Meaning
`[a-z]`	Lowercase letters	`[a-zA-Z]`	Uppercase and lowercase letters
`[A-Z]`	Uppercase letters	`[A-Z0-9]`	Uppercase letters and digits
`[0-9]`	Digits		

Examples

```
let pattern = /[A-Z]/;
```

- Matches any uppercase letter.

```
let pattern = /[0-9]/;
```

- Matches any digit from 0 to 9.

Matching Digits

- The \d metacharacter is used to match a digit.

Syntax: `\d`

It matches:

0 1 2 3 4 5 6 7 8 9

Example:

```
let text = "Student123";
console.log(/\d/.test(text));
```

Output: `true`

Equivalent Pattern:

`[0-9]` and `\d` both represent a digit.

5.1 Regular Expression(Continue..)

Character Classes

Entering a Range of Characters

- A hyphen **-** is used to specify a range.

Common Ranges

Pattern	Meaning	Pattern	Meaning
[a-z]	Lowercase letters	[a-zA-Z]	Uppercase and lowercase letters
[A-Z]	Uppercase letters	[A-Z0-9]	Uppercase letters and digits
[0-9]	Digits		

Examples

```
let pattern = /[A-Z]/;
```

- Matches any uppercase letter.

```
let pattern = /[0-9]/;
```

- Matches any digit from 0 to 9.

Matching Digits

- The **\d** metacharacter is used to match a digit.

Syntax: \d

It matches:

0 1 2 3 4 5 6 7 8 9

Example:

```
let text = "Student123";
console.log(/\d/.test(text));
```

Output: true

Equivalent Pattern:

[0-9] and \d both represent a digit.

Matching Non-Digits

- The **\D** metacharacter is used to match a non-digit character.

Syntax: \D

Example:

```
let text = "ABC";
console.log(/\D/.test(text));
```

Output: true

Comparison

Pattern	Meaning
\d	Digit
\D	Non-digit

Remember

- Lowercase and uppercase have opposite meanings in many Regex shortcuts.

Matching Punctuation and Symbols

- Special characters can be matched using escape characters.
- Some special characters are:

. * + ? ^ $ ( ) [ ] { }

- To match them literally, use \.

Examples

To search for a dot: /\./

To search for a question mark: /\?/

To search for a plus sign: /\+/

Example

```
let text = "Price: ₹500.";
console.log(/\./.test(text));
```

Output: true

5.1 Regular Expression(Continue..)

Character Classes

Matching Words

- The \w metacharacter matches a word character.
- It generally matches:

A-Z a-z 0-9 _

Syntax \w

Example

```
let text = "Hello";
console.log(/\w/.test(text));
```

Output true

Matching Non-Digits

- \W matches a character that is not a word character.

Syntax \W

Example

```
let text = "ABC";
console.log(/\W/.test(text));
```

Output true

Non-word Character

- \W matches a character that is not a word character.

Pattern	Meaning
\w	Word character
\W	Non-word character

Word Boundary

- The \b represents a word boundary.
- It is useful when we want to match a complete word.

Example

```
let text = "JavaScript is easy";
console.log(/\bJavaScript\b/.test(text));
```

Output true

Why use \b?

- It helps distinguish a complete word from a part of another word.

Replacing Text Using Regular Expressions

- The replace() method is used to replace matching text.

Syntax `string.replace(pattern, replacement);`

Example

```
let text = "I like Java.";
let result = text.replace(/Java/, "JavaScript");
console.log(result);
```

Output I like JavaScript.

Common Regular Expression Symbols

Symbol	Meaning	Example
.	Any character	/c.t/
\d	Digit	/\d/
\D	Non-digit	/\D/
\w	Word character	/\w/
\W	Non-word character	/\W/
\s	Whitespace	/\s/
\S	Non-whitespace	/\S/
\b	Word boundary	/\bcat\b/
^	Beginning	/^Hello/
$	End	/world$/

Important Point

- replace() returns a new string.
- The original string is not directly modified.

Prepared by-Prof. Priti Chavhan.

5.1 Regular Expression (Continue..)

Character Classes

Replacing Multiple Occurrences

The **g** flag means **global search.**

Example

```
let text = "cat dog cat";
let result = text.replace(/cat/g, "lion");
console.log(result);
```

Output:

lion dog lion

Without g

```
text.replace(/cat/, "lion");
```

- Only the first matching occurrence is replaced.

With g

```
text.replace(/cat/g, "lion");
```

- All matching occurrences are replaced.

Regular Expression Flags

Flags modify how a regular expression behaves.

Flag	Meaning
g	Global search
i	Case-insensitive search
m	Multiline search

Example

```
let pattern = /hello/i;
```

Matches:

- hello
- Hello
- HELLO

Global + Case-insensitive

```
let pattern = /hello/gi;
```

Returning the Matched Text

The **match()** method returns the text that matches a regular expression.

Syntax

```
string.match(pattern);
```

Example

```
let text = "My marks are 85";
let result = text.match(/\d+/);
console.log(result);
```

The matched number is:

85

\d+

- Means one or more digits.

5.1 Regular Expression (Continue..)

Character Classes

RegExp Object

JavaScript provides a built-in RegExp object for working with regular expressions.

Example

```
let pattern = new RegExp("JavaScript");
console.log(pattern.test("I am learning JavaScript"));
```

Output:

true

Common RegExp Methods

Method	Purpose
test()	Checks whether a match exists
exec()	Searches and returns match information

RegExp Object Properties

Important properties of a RegExp object include:

Property	Meaning
source	Returns the regular expression pattern
global	Indicates whether g flag is used
ignoreCase	Indicates whether i flag is used
multiline	Indicates whether m flag is used
flags	Returns the flags used
lastIndex	Position from which the next match starts for certain global/sticky operations

Example

```
let pattern = /hello/gi;
console.log(pattern.source);
console.log(pattern.global);
console.log(pattern.ignoreCase);
console.log(pattern.flags);
```

Output:

hello
true
true
gi

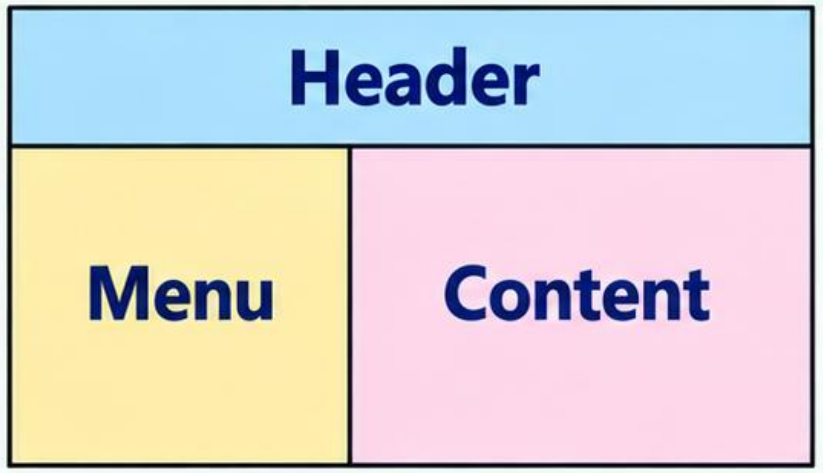
5.2 FRAMES

Introduction to Frames

What are Frames?

- A frame divides a browser window into **multiple sections.**
- Each section can display a **different HTML document.**

Example Layout:



Uses of Frames

1. Navigation menus
2. Displaying multiple documents
3. Creating parent-child browser layouts

Creating a Frame

Traditional frames were created using `<frameset>` and `<frame>`.

Example:

```
<frameset cols="30%,70%">
    <frame src="menu.html">
    <frame src="content.html">
</frameset>
```

Meaning:

- **30%** → menu.html
- **70%** → content.html
- The browser window is divided into two columns.

5.2 FRAMES

Introduction to Frames(Continue..)

Horizontal Frames

- Frames can also be arranged **horizontally**.
- The browser window is divided into top and bottom sections.

Example:

```
<frameset rows="30%,70%">
      <frame src="header.html">
      <frame src="content.html">
</frameset>
```

Layout:

Invisible Borders of Frame

- The **frameborder** attribute was traditionally used to control the **frame border**.
- It can be set to **0** to make the borders invisible.

Example:

```
<frameset cols="30%,70%" frameborder="0">
      <frame src="menu.html">
      <frame src="content.html">
</frameset>
```

Meaning:

- `frameborder="0"` means no visible border between frames.
- This is a legacy frames technique and is mainly relevant when studying older HTML/JavaScript syllabi.

5.2 FRAMES

Introduction to Frames(Continue..)

1 Calling a Child Window

JavaScript can open a new browser window using **window.open()**.

Syntax:

```
window.open(URL, name, features);
```

Example:

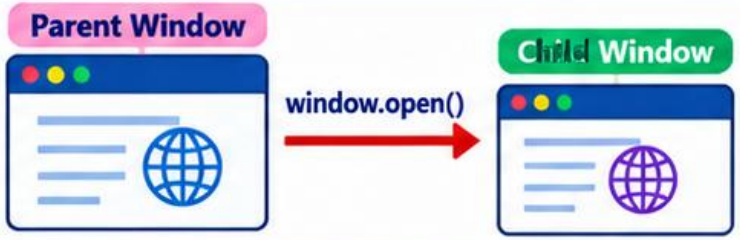
```
let child = window.open(
    "child.html",
    "childWindow",
    "width=500,height=400"
);
```

Here:

- child.html → page to open
- childWindow → name of window
- width=500 → width
- height=400 → height

2 Parent and Child Window

When one window opens another window:



The parent can keep a reference to the child.

Example:

```
let childWindow = window.open(
    "child.html",
    "child",
    "width=400,height=300"
);
```

Now **childWindow** refers to the newly opened window.

3 Changing Content of a Child Window

The parent window can change the location of the child window.

Example:

```
let child = window.open("page1.html");

child.location.href = "page2.html";
```

Meaning:

The child window changes from:

4 Giving Focus to a Child Window

The **focus()** method brings a child window to the front.

Example:

```
let child = window.open("child.html");

child.focus();
```

Purpose:

- If the child window is already open or behind another window, **child.focus()** attempts to bring it to the foreground.

5 Writing to a Child Window

JavaScript can write content into a newly opened window.

Example:

```
let child = window.open("", "child");

child.document.write("<h1>Hello Student</h1>");
```

Output in Child Window:

Important: The document object represents the HTML document displayed inside the child window.

5.2 FRAMES

Introduction to Frames(Continue..)

Accessing Elements of Another Child Window

1 Concept

- A parent window can access elements in a child window using its reference.

2 Parent Page (parent.html)

```
let child = window.open("child.html");

child.document.getElementById("msg").
innerHTML = "Hello Student";
```

3 Child Page (child.html)

```
<p id="msg">Original Message</p>
```

4 How It Works?

5 Output in Child Window

Important

- Access can be restricted by same-origin security rules when the parent and child pages come from different origins.

5.3 ROLLOVER

What is Rollover?

- A rollover is a web effect where an element changes its appearance when the user moves the mouse over it.
- It is also called a mouse-over effect.

Uses of Rollover

- Navigation menus
- Buttons
- Images
- Links
- Interactive web pages

Example

```
<button
  onmouseover="this.style.
  background='yellow'">
  onmouseout="this.style.
  background=''">
  Home
</button>
```

Example

Normal State	Mouse Over
Home	HOME

Creating Rollover

- JavaScript mouse events can be used to create a rollover.
- Important events:

onmouseover | onmouseout

Working

1. Mouse enters
2. onmouseover
3. Button changes
4. Mouse leaves
5. onmouseout
6. Original appearance

Prepared by-Prof. Priti Chavhan.

5.3 ROLLOVER(Continue..)

1 Image Rollover

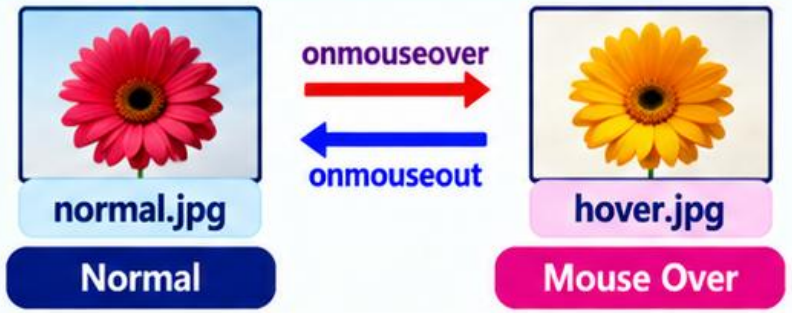
- An image can be changed when the mouse moves over it.
- This is called an **image rollover**.

Example:

```
<img src="normal.jpg"
     onmouseover="this.src='hover.jpg'"
     onmouseout="this.src='normal.jpg'">
```

Working:

Event	Image
Normal	normal.jpg
Mouse Over	hover.jpg
Mouse Out	normal.jpg



2 Text Rollover

- Text can also change when the mouse pointer moves over it.

Example:

```
<p onmouseover="this.style.color='red'"
   onmouseout="this.style.color='black'>
    Move mouse over this text
</p>
```

Result:

Before mouse-over:

Move mouse over this text

During mouse-over:

Move mouse over this text

3 Multiple Actions for Rollover

- A rollover can perform more than one action.
- For example, when the mouse moves over a button:
  - Change background color
  - Change text color
  - Change font size

Example:

```
<button onmouseover="
    this.style.background='blue';
    this.style.color='white';
    this.style.fontSize='20px';
  "
   onmouseout="
    this.style.background='';
    this.style.color='';
    this.style.fontSize='';
  ">
   Click Here
</button>
```

Multiple Actions:

Mouse Over
↓
Background changes
+
Text color changes
+
Font size changes

Thank You!

I hope this session has helped you to understand **basic concepts of Javascript** and its **basic usage.**

Keep Learning, Keep Coding!

Prepared by – *Prof. Priti Chavhan*

Unit No. 6

Menus, navigation and web page protection

Department of Computer Engineering
Dept. of Polytechnic & Skill Development

Course Outcome

- CO6- Create Menus and navigations in web pages.

Content:

6.1 Status bar- builds a static message, changing the message using rollover, moving the message along the status bar

6.2 Banner - loading and displaying banner advertisement. Linking a banner advertisement to URL

6.3 Slide Show - creating a slide show

6.4 Menus-creating a pulldown menu, dynamically changing a menu, validating menu selection, Floating menu, chain select menu, tab menu, pop-up menu, sliding menu, highlighted menu, folding a tree menu, context menu, scrollable menu, side bar menu.

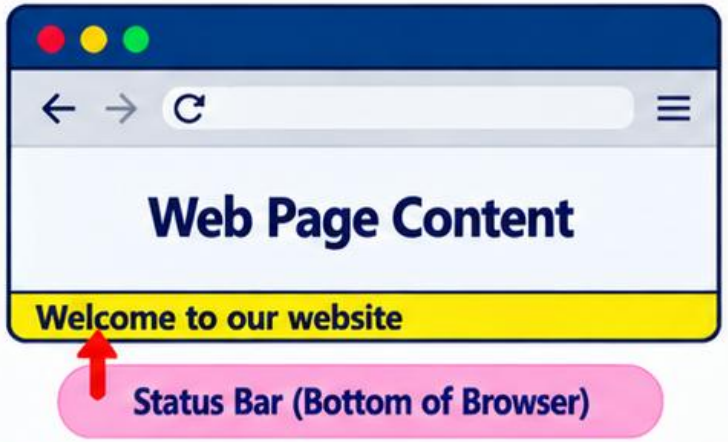
6.5 Protecting web page - hiding your code, disabling the right mouse button, JavaScript, concealing email address.

6.6 Frameworks of JavaScript and its application

6.1 STATUS BAR

1 What is a Status Bar?

- Status bar is the area at the bottom of a browser window.
- JavaScript can traditionally be used to display messages in it.



2 Three Operations

1. Static message
2. Change message using rollover
3. Move message along the status bar

3 Static Message

- A fixed message can be displayed in the browser's status bar.

Example:

```
<body onload="window.status='Welcome';">
```

Working:

Page Loads
↓
JavaScript Executes
↓
Message Displayed in Status Bar

Note:

- Modern browsers may restrict `window.status`, so this is mainly relevant to older/syllabus-based JavaScript.

4 Status Bar Using Rollover

- A message can change when the mouse moves over an element.

Example:

```
<a href="#"
   onmouseover="window.status='Click here';"
   onmouseout="window.status='';">
   Continue
</a>
```

Working:

Mouse Over
onmouseover
↓
Message changes

Mouse Out
onmouseout
↓
Message clears

5 Moving Message Along the Status Bar

- A moving message can be created using:

JavaScript Timer + String Manipulation

Prepared by-Prof. Priti Chavhan.

6.2 BANNER

1 What is a Banner?

- A banner is a rectangular image or advertisement displayed on a web page.

Example:

Uses:

- Advertisements
- Promotions
- Offers
- Announcements

2 Displaying a Banner

- A banner image can be displayed on a web page.

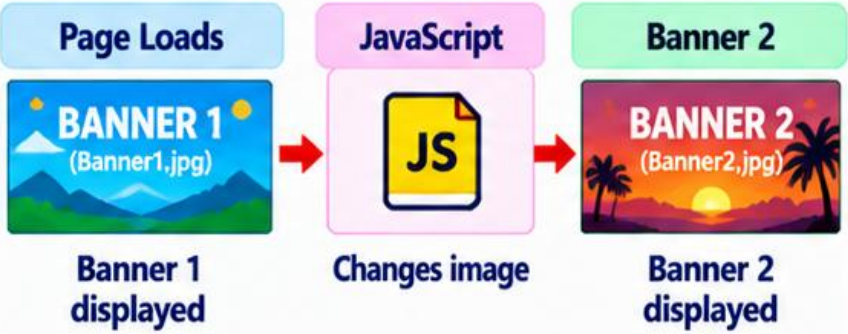
HTML Code:

```
<img id="banner" src="banner1.jpg">
```

Change Banner using JavaScript:

```
document.getElementById("banner").
 src = "banner2.jpg";
```

Working:



Page Loads → Banner 1 → JavaScript → Banner 2

3 Linking a Banner Advertisement

- A banner can be made clickable by linking it to a URL.

Example:

```
<a href="https://www.example.com">
    <img src="banner.jpg">
</a>
```

Working:

User clicks banner
↓
Browser follows the URL
↓
Target website opens

Remember:

- Banner + Hyperlink = Clickable Advertisement

Prepared by-Prof. Priti Chavhan.

6.3 SLIDE SHOW

What is a Slide Show?

- A slide show displays a series of images one after another.

JavaScript

- Store image names in an array and create a function to change the image.

```
let images = [
    "image1.jpg",
    "image2.jpg",
    "image3.jpg"
];
let i = 0;

function nextSlide() {
   i = (i + 1) % images.length;
   document.getElementById("slide").src = images[i];
}
```

Basic HTML

- Use an <img> tag to show the image on the web page.

```
<img id="slide" src="image1.jpg">
```

Automatic Slide Show

- Change slides automatically using setInterval().

```
setInterval(nextSlide, 3000);
```

3000 milliseconds = 3 seconds

- So, the image changes every 3 seconds.

Important Functions

Function	Purpose
nextSlide()	Changes the image
setInterval()	Repeats an action
getElementById()	Accesses an HTML element

6.4 MENUS

What is a Menu?

- A menu provides a list of options for navigation or performing actions.

Main Purpose

- Menus make websites easier to navigate.

Common Menu Types

Pull-down Menu

Select
Home
About
Contact

Dynamic Menu

Courses
CSE
IT
Mechanical

Chain Select Menu

Select State
Maharashtra
Select City
Pune
Mumbai
Nagpur

Tab Menu

Home | Gallery | Contact

Welcome to Home Page

Pop-up Menu

Open
Save
Delete
Properties

Sliding Menu

Home
Profile
Settings

Highlighted Menu

Home
About
Services
Contact

Dynamic & Chain Select Menus

Dynamic Menu

- JavaScript can change menu options while the page is running.

Example:

Select Branch → CSE | IT → Select Subject → JavaScript | Database | Python

Chain Select Menu

- One menu depends on the selection made in another menu.

Example:

Select State → Maharashtra → Select City → Pune | Mumbai | Nagpur

Key Point: Selection in one menu determines the options in the next menu.

Validating Menu Selection

What is Validation?

- Validation checks whether the user has selected a valid option.

HTML (Drop-down Menu)

```
<select id="course">
  <option value="">Select Course</option>
  <option value="cse">CSE</option>
  <option value="it">IT</option>
</select>
```

JavaScript (Validation)

```
if (document.getElementById("course").value == "") {
    alert("Please select a course");
}
```

Working

Submit → Check Selection → Empty ? → Show Error

Purpose

- Prevent invalid input
- Ensure required selection
- Improve user experience

6.4 MENUS

Tab Menu

- Organizes content into separate tabs.

Pop-up Menu

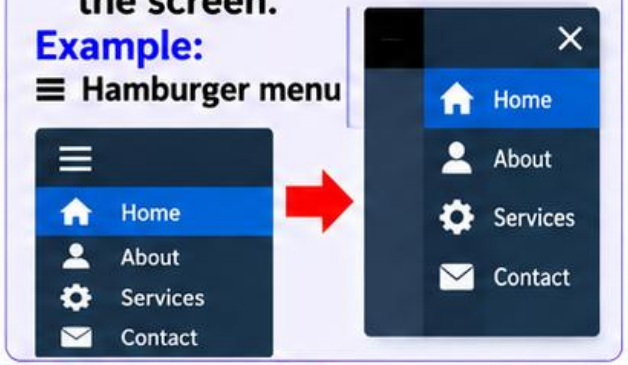
- Appears when the user performs an action such as clicking a button.

Sliding Menu

- Moves into or out of the screen.

Example:

Hamburger menu



Advantages

- Saves screen space
- Organizes information
- Provides easy navigation

Floating, Highlighted, Tree & Sidebar Menus

Floating Menu

- Remains visible while the page is scrolled.

Highlighted Menu

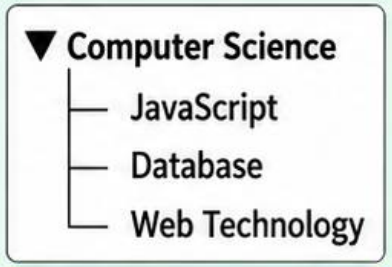
- Changes appearance during user interaction.

Common events:

- onmouseover
- onmouseout

Folding Tree Menu

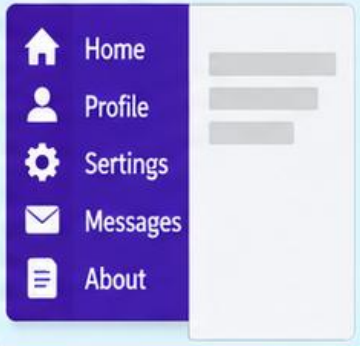
- Displays information hierarchically.



Click → Expand / Collapse

Sidebar Menu

- Appears on the left or right side of a web page.



Context & Scrollable Menus

Context Menu

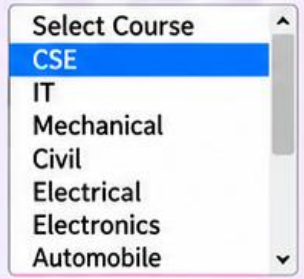
- Appears when the user performs a right-click.

Event: contextmenu

```
<body oncontextmenu=
  "return false;">
```

Scrollable Menu

- Displays many options inside a limited area.



Uses:

- Long lists
- Mobile applications
- Large navigation menus

Remember: A context menu can be disabled using JavaScript, but this is not a reliable security mechanism.

6.5 PROTECTING WEB PAGE

What is Web Page Protection?

- Techniques used to make copying or accessing certain page information more difficult.

Techniques

1. Hiding / Obfuscating Code
2. Disabling Right Mouse Button
3. JavaScript-based Protection
4. Concealing Email Address

Important

- JavaScript protection provides only **basic deterrence**, **not complete security.**

⚠ These techniques do not provide complete protection.

1. Hiding Code

Minification / Obfuscation

- Code is rewritten in a shorter and less readable form.

Original Code:

```
let studentName = "Rahul";
console.log(studentName);
```

Minified / Obfuscated Code:

```
let a="Rahul";console.log(a);
```

Remember: Minification ≠ Encryption.

2. JavaScript Protection

Disabling Right Mouse Button

```
<body oncontextmenu="return false;">
```

- It prevents the normal context menu from appearing.
- **But:** Users can still use developer tools and other methods.

3. Concealing Email Address

- Instead of directly displaying:

student@example.com

- JavaScript can construct it:

```
let user = "student";
let domain = "example.com";
document.write(user + "@" + domain);
```

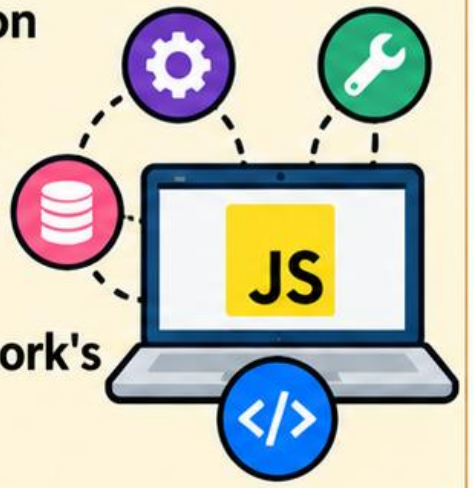
Important Points

- These techniques **do not provide complete protection.**
- **Never** store **passwords or secret keys** in client-side JavaScript.

6.6 FRAMEWORKS OF JAVASCRIPT AND THEIR APPLICATION

What is a JavaScript Framework?

- A JavaScript framework is a collection of **tools, conventions**, and **reusable features** that helps developers build web applications more efficiently.
- Instead of writing everything from scratch, developers can use a framework's structure and features.



Simple Idea

Without Framework

Developer

↓

Write most things from scratch

↓

Build Application

With Framework

Developer

↓

Framework + Reusable Features

↓

Build Application Faster

Popular JavaScript Frameworks

Framework	Common Application
Angular	Large web applications
React*	User interfaces and web applications
Vue.js	Interactive web interfaces
Svelte	Modern web interfaces

Note about React

- React is technically a JavaScript library, although it is often discussed alongside frameworks in introductory web-development courses.

6.6 FRAMEWORKS OF JAVASCRIPT AND THEIR APPLICATION

Angular

ANGULAR

What is Angular?

- Angular is a **web development framework** based on **JavaScript/TypeScript**.
- It is developed and maintained by **Google** and the open-source community.
- It is useful for building **large, structured** web applications.

Applications of Angular

- Enterprise applications
- Dashboards
- Single Page Applications
- E-commerce applications

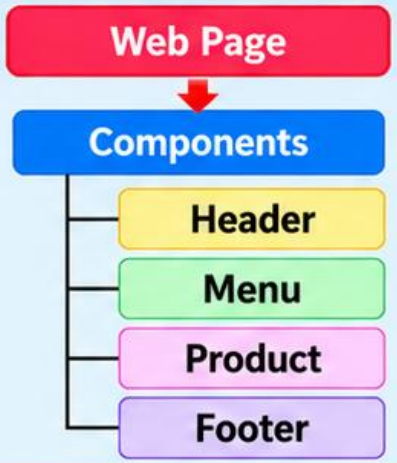
React

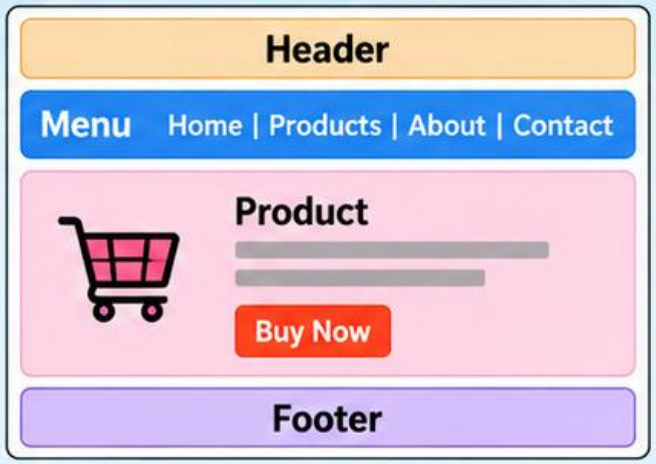
React

What is React?

- React is a **JavaScript library** for building user interfaces.
- It uses **reusable components**.

Example: Components in a Web Page





Applications of React

- Interactive websites
- Web applications
- E-commerce interfaces
- Single Page Applications

6.6 FRAMEWORKS OF JAVASCRIPT AND THEIR APPLICATION

Vue.js

Vue.js

What is Vue.js?

- Vue.js is a progressive JavaScript framework for building user interfaces.
- It is designed to be approachable and component-based.

Applications of Vue.js

- Interactive websites
- Dashboards
- Single Page Applications
- Dynamic forms

Build interactive and dynamic web interfaces

Svelte

Svelte

What is Svelte?

- Svelte is a modern framework for building web interfaces.
- It shifts much of the work to the build step, producing efficient browser code.

Applications of Svelte

- Interactive websites
- Web applications
- User interfaces

Modern and efficient web interfaces

JavaScript Framework – Advantages

1. **Reusability**
   Components can be reused.
2. **Faster Development**
   Developers don't need to build everything from scratch.
3. **Better Organization**
   Applications can be divided into components/modules.
4. **Maintainability**
   Well-structured code is easier to update.
5. **Interactive UI**
   Frameworks make it easier to create dynamic user interfaces.

Thank You!

I hope this session has helped you to understand **basic concepts of Javascript** and its **basic usage.**

Keep Learning, Keep Coding!

Prepared by – *Prof. Priti Chavhan*